Planning and Approximate Reasoning (PAR)

Final Practical Project

Autonomous Drone Planning for Smart-City Emergency Response

Sonia Castro Paniello

Michelle Miranda

Master in Artificial Intelligence

November 2026

Contents

1 Introduction **2**

2 Problem Analysis **2**
2.1 State and search space . . . 2
2.2 Types: what never changes . . . 3
2.3 Avoiding negative preconditions . . . 3
2.4 Locked locations and unlocking . . . 3
2.5 Symbolic battery and plan cost . . . 3
2.6 Actions and goal . . . 4

3 PDDL Implementation **4**

4 Alternative Models **5**

5 Problem Instances **6**
5.1 Small instance: Barcelona flood . . . 7
5.2 Medium instance: Barcelona storm surge . . . 7
5.3 Large instance: Tarragona chemical leak . . . 8

6 Planner Setup **9**

7 Results and Analysis **10**
7.1 Small instance . . . 10
7.2 Medium instance . . . 12
7.3 Large instance . . . 13
7.4 Single-drone comparison and clustered start . . . 14
7.5 Across instances . . . 14

8 Literature Connection **15**

9 Limitations and Future Work **15**

A Appendix **16**
A.1 Domain files . . . 16
A.2 Problem files . . . 22
A.3 Example plans . . . 29
A.4 VAL outputs . . . 30

1 Introduction

After an emergency in a city, a fleet of autonomous drones has to inspect damaged locations and bring supplies such as medical kits, water and communication equipment from depots to the affected locations. Each drone carries one item at a time, and its battery limits how far it can fly before it needs a charging station. Some areas are blocked at first and only become reachable once a nearby site has been inspected or has received its supply. The goal is to find a sequence of flights, inspections, pick-ups, deliveries and recharges that meets every demand at the lowest total cost.

In this project we model the scenario in PDDL and solve it with Fast Downward. Since the planner does not support numeric fluents, the battery is encoded with symbolic levels. We build two alternative models that differ only in how recharging works: a full recharge in one action (Model A), or one level per action (Model B). We test both on a small, a medium and a large instance with several planner configurations, validate every plan with VAL, and compare plan quality and search effort.

2 Problem Analysis

We built the model from the problem statement. This section covers what we decided and learned while building it and comparing it with the templates.

2.1 State and search space

A state is the set of facts that are true at a given moment. The planner only matches facts and does not know what a predicate means, so everything the search needs must be written explicitly. We split the facts into two groups:

- **Static facts**, written once in `:init` and never changed: the map (`connected`, `flight_cost`, `distance`), the chargers, the unlock relations and the battery arithmetic tables.
- **Dynamic facts**, changed by actions: drone position, battery level, cargo, supply positions, accessibility, pending inspections and pending deliveries.

The search space is the set of reachable combinations of dynamic facts. An approximate **upper bound** on its size is

$$(L\,(N{+}1))^{D} \cdot (D{+}2)^{S} \cdot 2^{I},$$

where D is the number of drones, L the number of locations, $b_0 \dots b_N$ the battery levels, S the number of supplies and I the number of inspection goals. The first term counts, for every drone, its position (L values) and its battery level ($N{+}1$ values). The second term gives each supply $D{+}2$ possible states: still at its depot, delivered, or carried by one of the D drones. The last term records whether each inspection has been done. The exponent D is why adding drones costs much more than adding locations, which the large instance shows clearly (Section 7). Table 1 gives the size of the grounded task that Fast Downward builds for each instance. These numbers are identical in both models.

	Small	Medium	Large (1 drone)	Large (3 drones)
Grounded facts	23	62	56	103
Grounded actions	40	316	196	588

Table 1: Size of the grounded task (Fast Downward translator), same in Model A and B.

2.2 Types: what never changes

The base types are `drone`, `location`, `supply` and `level`. Supply kinds are **subtypes** (`medical_kit`, `water_package`, `communication_device`): the kind of a supply never changes and every supply has exactly one, so it fits a type. Location roles (charging station, mission site, locked) are **predicates** instead: one location can play several roles at once, and some roles change during the plan, which a type cannot do. We added no `depot` predicate, following one rule: a predicate is only needed if a precondition or the goal tests it. No action asks whether a location is a depot. The depot is simply where the supplies start (`supply_at` in `:init`).

2.3 Avoiding negative preconditions

Several conditions are naturally negative: "the drone carries nothing", "not inspected yet", "not locked", "battery not full". Each one is replaced by a positive fact that actions add and delete, so the domain only needs STRIPS, typing, conditional effects and action costs:

- `cargo_empty`: "carrying nothing" written as a negation would also need a quantifier over every supply.
- `needs_inspection`: `inspect` requires it and deletes it. A second inspection then has no matching fact, and locations nobody asked about (depot, charger) cannot be inspected at all, so there are fewer grounded actions. `inspected` is kept as a separate record because the goal checks it.
- `accessible` instead of `locked`: it is true in `:init` for every location except the locked ones, including every drone start location.

2.4 Locked locations and unlocking

"Locked" is not the same as "needs inspection". A mission site must be enterable, otherwise the drone could never reach it to inspect it. A locked location cannot be entered until one specific event happens: inspecting some location X, or delivering some supply S. The problem designer chooses these pairs, so they are static facts in the problem file: `unlocked_by_inspection(X, Y)` and `unlocked_by_delivery(S, Y)`.

Delivery is keyed by the supply, not by the location. If it was only keyed by location, if a site needed two supplies, the first one would open the door. Adding the location as an extra argument is redundant: each supply object is needed by exactly one location, so the supply already fixes where it is delivered.

Rejected alternative: blocking the exit of the unlocking location (a precondition to fly along the lines of "a drone at X can only leave if X is inspected"). It depends on the map shape, it traps drones that only pass through X, and it cannot express a delivery that unlocks a location. We kept the check on the *destination*, `(accessible ?to)` in `fly`.

2.5 Symbolic battery and plan cost

Fast Downward has no numeric fluents, so the battery must be symbolic. Our first idea was one predicate per battery level (`batteryb1(drone)` ... `batterybn(drone)`), which needs a separate `fly` action for every (start level, cost) pair and changes the domain with every capacity. Instead, battery levels are objects $b_0 \dots b_N$ and the current level is `(battery ?d ?l)`, as in the template. One `fly` action then covers every level, and the domain does not depend on the capacity.

We separate `connected` (the route exists) from `flight_cost` (how many levels it consumes, as a level object). The plan cost needs a number: `(increase (total-cost) ?c)` is invalid, because `b2` is a name and not a number, and the planner cannot convert one into the other. The same cost is therefore stored again in the static numeric function `distance`. `fly` adds the

distance to `total-cost` and every other action adds 1. The two values must match: otherwise the battery drains one amount while the plan cost counts another, and VAL still accepts the plan. We checked both lists against the location diagrams, and every edge is written in both directions, because the planner only matches facts that are declared.

The planner knows nothing about the order of the levels, so subtraction is a static lookup table `battery_after(before, cost, left)`, with rows only where *before* $\geq$ *cost*. A flight with too little battery has no matching row, so it is never applicable. The table also fixes `?left`: without it, `?left` would be a free parameter and a drone could gain battery while flying. Illegal movements are automatically excluded by the absence of a matching fact.

2.6 Actions and goal

The five actions are `fly`, `inspect`, `pick-up-supply`, `deliver-supply` and `recharge`. `deliver-supply` does not put the supply back on the map at the destination. Otherwise another empty drone could pick it up again, which is unrealistic (the supply is used up) and adds pointless branches to the search. The goal contains only the mission (`inspected`, `delivered`), not `accessible`: unlocking is a means, not a mission.

3 PDDL Implementation

Both domains (`domain-A.pddl`, `domain-B.pddl`) are fully commented and listed in Appendix A. They differ only in `recharge` (Section 4). Both are named `drone_emergency`, so the same problem files serve both models without changes.

Requirements. `:strips` (add and delete effects), `:typing`, `:conditional-effects` (the `forall`/`when` unlocking) and `:action-costs` (`total-cost`, `distance`). We do not need `:negative-preconditions`.

Predicates. Table 2 lists every predicate and its meaning.

Predicate	Meaning	S/D
`drone_at ?d-drone ?l-location`	drone `?d` is at location `?l`	D
`carrying ?d-drone ?s-supply`	drone `?d` carries supply `?s`	D
`cargo_empty ?d-drone`	`?d` carries nothing*	D
`connected ?from ?to-location`	direct flight possible (both directions declared)	S
`has_charging_station ?l-location`	`?l` has a charger	S
`accessible ?l-location`	`?l` can currently be entered	D
`unlocked_by_inspection ?x ?y-location`	inspecting `?x` makes `?y` accessible	S
`unlocked_by_delivery ?s-supply ?y-location`	delivering supply `?s` makes `?y` accessible	S
`needs_inspection ?l-location`	`?l` still has to be inspected	D
`inspected ?l-location`	`?l` has been inspected (goal)	D
`needs_supply ?l-location ?s-supply`	`?l` is waiting for supply object `?s`	D
`supply_at ?s-supply ?l-location`	`?s` lies at `?l` and is not carried	D
`delivered ?s-supply ?l-location`	`?s` has been delivered to `?l` (goal)	D
`battery ?d-drone ?b-level`	current battery level of `?d`	D
`flight_cost ?from ?to-location ?c-level`	the flight costs `?c` levels	S
`battery_after ?b ?c ?left-level`	$b - c = \mathit{left}$, only listed if $b \geq c$	S
`next_level ?b ?up-level`	$\mathit{up} = b + 1$, no row starts at max	S
`max_level ?b-level`	full capacity (read by Model A only)	S

*Each drone carries at most one supply at a time.

Table 2: Predicates (S = static, D = dynamic). Functions: `total-cost` and static `distance ?from ?to-location`.

Actions. Table 3 summarises every action and its meaning.

Action	Preconditions	Effects	Cost
`fly`	at origin; connected; destination accessible; current battery; flight cost; `battery_after` row exists	move drone; replace battery level by the remaining one	distance
`inspect`	drone at `?l`; `needs_inspection ?l`	add `inspected`; delete `needs_inspection`; unlock every `?y` with `unlocked_by_inspection ?l ?y`	1
`pick-up-supply`	drone and supply at the same location; `cargo_empty`	delete `supply_at` and `cargo_empty`; add `carrying`	1
`deliver-supply`	drone at `?l`; `carrying ?s`; `needs_supply ?l ?s`	delete `carrying` and `needs_supply`; add `cargo_empty` and `delivered`; unlock every `?y` with `unlocked_by_delivery ?s ?y`	1
`recharge`	drone at a charger; current level has a `next_level`	Model A: level $\rightarrow$ max. Model B: level $\rightarrow$ level $+1$	1

Table 3: Actions of both domains.

The `fly` action shows how the battery encoding works: each precondition fixes one unknown parameter. `(battery ?d ?b)` takes the current level from the state, `flight_cost` takes the cost from the map, and `battery_after` takes the remaining level from the table. Nobody passes parameter values, the planner keeps only the combinations for which every precondition is a true fact.

```
(:action fly
  :parameters (?drone - drone ?from_location ?to_location - location
               ?battery_before ?cost ?battery_left - level)
  :precondition (and (drone_at ?drone ?from_location)
    (connected ?from_location ?to_location)
    (accessible ?to_location) ; not locked
    (battery ?drone ?battery_before)
    (flight_cost ?from_location ?to_location ?cost)
    (battery_after ?battery_before ?cost ?battery_left)) ; enough battery
  :effect (and (not (drone_at ?drone ?from_location)) (drone_at ?drone ?to_location)
    (not (battery ?drone ?battery_before)) (battery ?drone ?battery_left)
    (increase (total-cost) (distance ?from_location ?to_location))))
```

Unlocking uses a conditional effect, in `inspect` and the same way in `deliver-supply`: "for every location y, if this event opens y, add `(accessible y)`". `?locked_location` exists only inside the `forall`, it is not an action parameter. If no unlock fact matches, nothing extra happens.

```
(forall (?locked_location - location)
  (when (unlocked_by_inspection ?location ?locked_location)
        (accessible ?locked_location)))
```

4 Alternative Models

The two models differ only in `recharge`. In Model A (full recharge) one action sets the battery to `max_level`, whatever the current level, at cost 1. In Model B (partial recharge) each action raises the battery by one level through `next_level`, at cost 1.

An optimal planner never keeps an action that only adds cost like recharging at full batery, but it would still be grounded and tried. Model B blocks it for free, because no `next_level`

row starts at the maximum. In Model A we first used an extra static predicate `isnotmax`. We then noticed that the starting levels of the `next_level` rows are exactly that list, so Model A also requires `(next_level ?current ?higher)`, only as a "not full" check. This removes one predicate and lets both domains share the same problem files. `max_level` is simply ignored by Model B. Both `recharge` actions have the same number of groundings, which is why Table 1 is identical for the two models.

Table 4 summarises the evidence from the optimal configuration (A* with LM-cut), the full results are in Section 7.

	Optimal cost		Recharges / length		Expanded states (time)	
Instance	A	B	A	B	A	B
Small	10	10	1 / 10	1 / 10	16 (0.2 s)	16 (0.1 s)
Medium	26	31	1 / 24	6 / 29	123k (8.0 s)	336k (14.2 s)
Large (3 drones)	30	31	1 / 26	1 / 27	115k (58 s)	243k (123 s)
Large (1 drone)	38	50	4 / 33	16 / 45	4.5k (0.4 s)	11.4k (0.8 s)
Large clustered†	31	–	1 / 28	–	420k (203 s)	time limit

Table 4: Model A vs Model B, A* + LM-cut, 5 min / 4 GB. †Extra experiment (three drones starting at the same depot).

Expressiveness. Model B is more realistic as charging takes time roughly proportional to the energy restored. It also allows partial charging, so a drone can take just enough energy for the rest of its route. **Planning complexity.** The grounded tasks have the same size, so the difference comes from plan depth. In B the optimal plans are longer (up to 12 extra actions), and LM-cut expands about 2–3 times more states. We think part of the reason is the delete relaxation behind the heuristics: the starting battery fact is never deleted there. The more recharge actions a plan needs, the more the heuristic underestimates, and B needs many more. **Plan quality and runtime.** The optimal costs differ exactly by the extra recharge actions. In the medium instance and the single-drone large instance the flights are the same in both models (13 and 17), and B adds 5 and 12 recharges. When only one level is missing (small instance) the two models coincide. Runtime follows the expanded states, roughly doubling in B. **Trade-offs.** Model A gains shorter plans and faster search, at the cost of a less faithful cost model. Recharging is cheap whatever the battery level, so the planner has no incentive to save energy. Model B gains realism and plan costs that reflect energy use, at the cost of deeper search, which becomes the bottleneck when several drones need several levels each.

5 Problem Instances

Our master is taught on campuses in Barcelona and Tarragona, so we placed the instances in the two cities we travel between. The small and medium instances are flood emergencies in Barcelona, and the large one is a chemical emergency in Tarragona, where the URV campus is one of the charging stations. The positions of the locations follow the real geography loosely; edge costs, chargers and locks are our design choices. Each diagram shows the location graph with edge costs, depots, chargers, mission sites, drone starts with their battery level, the initial supply positions, and which event unlocks which location. Before running the experiments we checked every file against Section 5 of the statement: the counts, `connected`, `flight_cost` and `distance` symmetric and equal, complete `battery_after` tables, every supply starting at a depot, and every drone start accessible.

5.1 Small instance: Barcelona flood

A storm floods the Barceloneta district and cuts off Parc de la Ciutadella, used as an evacuation centre. One drone starts full (`b4`) at the El Prat aerodrome, the depot that holds `kit1` (Figure 1). The five locations form a line, all with cost 1: `hospital_viladecans` – `aerodrome` – `barceloneta` – `montjuic_pad` (charger) – `ciutadella`. The goals are to deliver `kit1` to Hospital de Viladecans, across the Llobregat, and to inspect Barceloneta and Ciutadella. Ciutadella is locked until Barceloneta has been inspected.

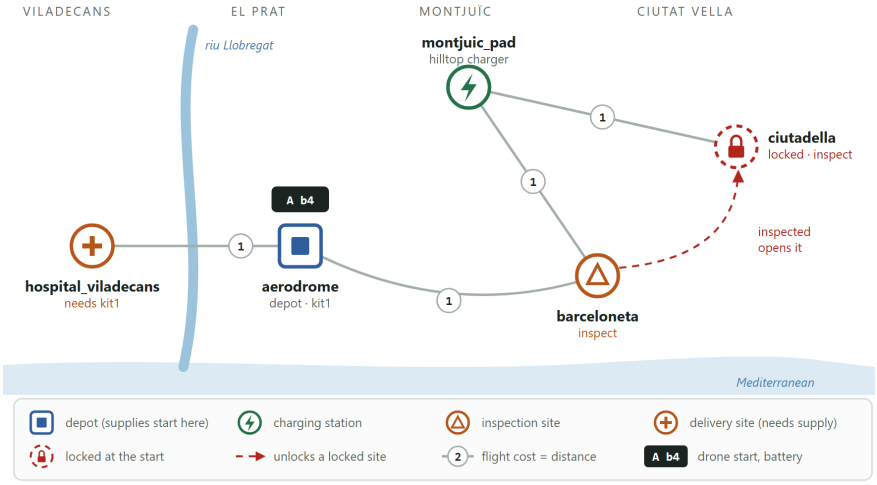


Figure 1: Small instance (Barcelona): one drone, five locations, every edge costs 1.

Why a recharge is unavoidable. The drone starts between the two ends of the line, and both ends are mission sites. It must reach one end and come back. The cheapest way is aerodrome → hospital → aerodrome (2 flights) and then aerodrome → barceloneta → montjuic_pad → ciutadella (3 flights). Every complete mission needs at least 5 flights, which is 5 levels, more than the 4 of a full battery. A breadth-first search over the map confirmed that 5 is the minimum. On the best route the drone reaches the only charger with `b0`, just before the last flight. The optimal cost is 10 in both models (5 flights, 2 inspections, pick-up, delivery and one recharge). In Model B a single `b0→b1` step is enough.

Getting here took three versions. In the first, the recharge was forced only by starting the drone at `b3`: with a full battery a 4-flight route existed. We then removed two edges to form a ring, but `kit1` was stored at the charger, not at the depot as the scenario requires. Moving it to the depot made the ring solvable in 3 flights. The line map meets both rules at once.

5.2 Medium instance: Barcelona storm surge

Flooding hits the coast and the Besòs basin (Figure 2). Two drones start at `port_depot` (`b5`) and at the `glories_tower` charger (`b4`). There are 8 locations: one depot holding the three supplies, two chargers (`montjuic_pad`, `glories_tower`) and five mission sites. The capacity is `b6` and edges cost 1 or 2. The goals are 4 inspections (Barceloneta, Hospital del Mar, Poblenou, Besòs delta) and 3 deliveries of three different kinds: `kit1` to the hospital, `water1` to Poblenou and `comm1` to the Fòrum shelter.

The locked site. `besos_delta` is not accessible at the start and opens when its neighbour `poblenou` is inspected. A sequence that fails is flying there first: `(fly droneB glories_tower besos_delta b4 b2 b2)`. The battery rows and the edge exist, but the precondition

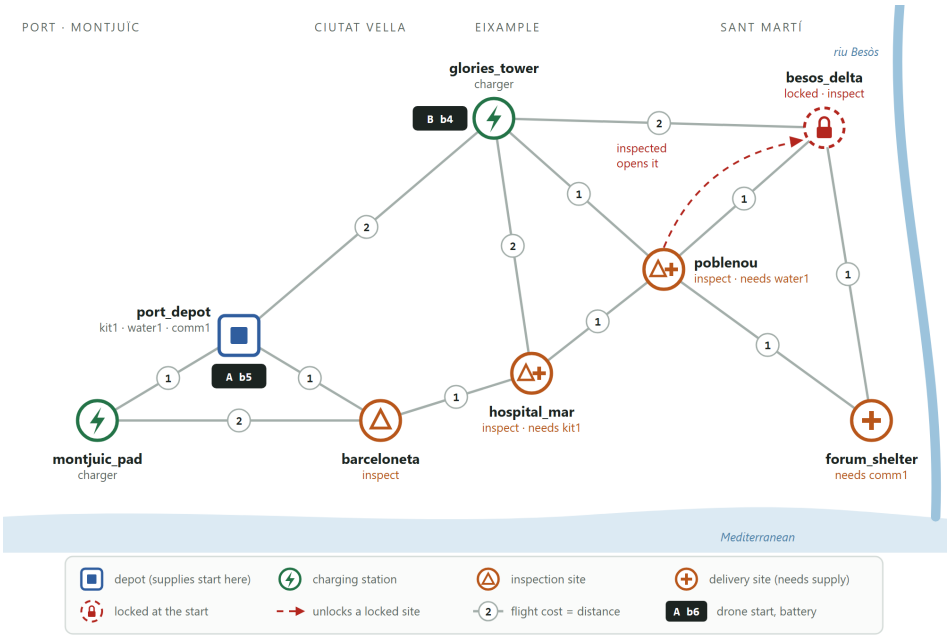


Figure 2: Medium instance (Barcelona): two drones, eight locations, edge costs 1–2.

`(accessible besos_delta)` of `fly` is false, so the action is not applicable, and every route to the site is blocked the same way. Note that it is not `unlocked_by_inspection` that blocks it: that is a static fact, true from the start, and never a precondition of `fly`. It only tells `inspect` what to add. Inspecting `poblenou` fires the conditional effect and adds `(accessible besos_delta)`.

5.3 Large instance: Tarragona chemical leak

After a leak at the petrochemical park, responders need a radio relay at Bonavista before drones may enter the park. The old town (Part Alta) stays closed until the Roman amphitheatre below it has been checked for damage (Figure 3). There are 12 locations: two depots (`port` with `comm1` and `water1`; `estacio` with `kit1` and `kit2`), three chargers (`campus`, `rambla`, and `bonavista`, which is also a delivery site), and mission sites on both banks of the Francolí. The capacity is `b6` and edges cost 1–3; the longest edge crosses the river. There are 8 goals: 4 deliveries of three kinds and 4 inspections.

Locks. The required delivery lock is `(unlocked_by_delivery comm1 petrochem)`: `comm1` must go from the port to Bonavista, and only then can a drone fly to the park and inspect it. It works like the inspection lock, but the conditional effect is in `deliver-supply` and the key is the supply. We added an inspection lock as well: `part_alta` opens after `amfiteatre` is inspected, so the `kit1` delivery also has an ordering constraint.

Single-drone variant. Following the hint in the statement, the three drones start full in different regions: `droneA` at the port (west bank), `droneB` at `rambla` (city centre) and `droneC` at `campus` (north). Each one can serve its own group of sites. `problem-large-1drone.pddl` keeps the same map, goals, supplies and locks with `droneA` only, which must cross the whole city and recharge more often. As an extra experiment, `problem-large-clustered.pddl` starts all three drones at the port. This separates the gain from having more drones from the gain from starting them in different places. Section 7 compares the costs.

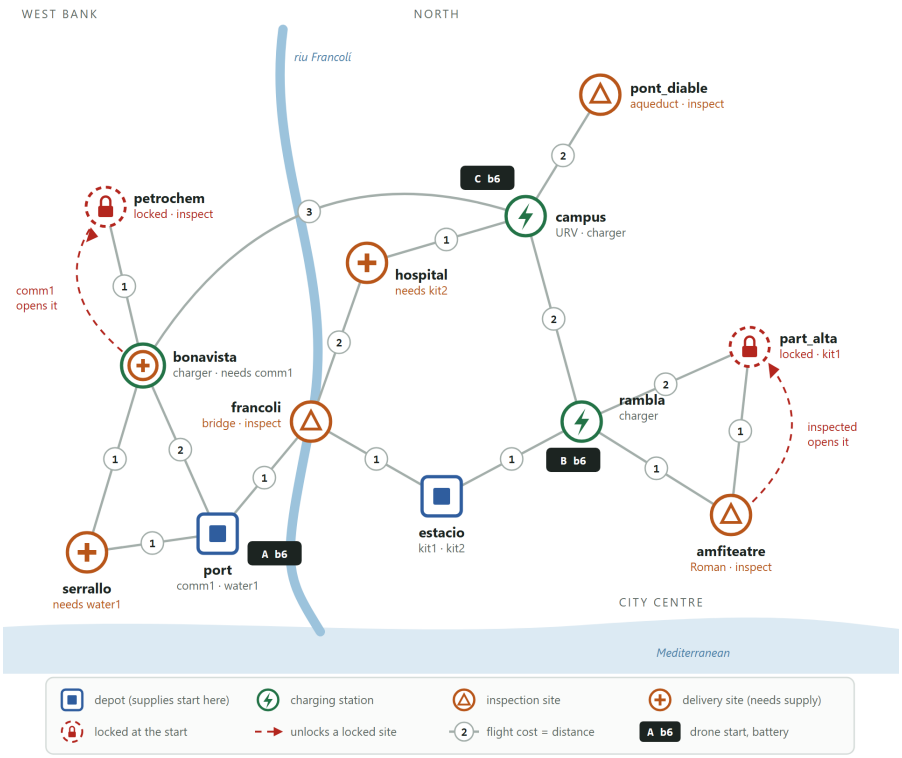


Figure 3: Large instance (Tarragona): three drones, twelve locations, edge costs 1–3, one delivery lock and one inspection lock.

6 Planner Setup

Tools. We built Fast Downward [1] (commit `487e33c`, October 2026) and VAL [2] (commit `3c7a1f3`) from source in WSL 2 (Ubuntu 24.04) on Windows 11, on a laptop with an Intel Core Ultra 5 115U and 7 GB of memory available to WSL. The exact commands are in `README.txt`.

Configurations. Table 5 lists the configurations. The two required ones are LAMA-first [4] (fast, satisficing) and A* with LM-cut [3] (cost-optimal). We added three more so we could see the effect of the heuristic separately from the effect of the search. A* with blind, h^{max} and LM-cut keeps the search fixed and uses heuristics that are increasingly informed (all admissible, so all three return optimal plans). `eager_greedy([ff()])` is a greedy search with plain h^{FF} and no landmarks or preferred operators, which shows what LAMA-first adds. The anytime `-alias lama` was run on the large variants, as the statement suggests, because LAMA-first's first plan can be far from the best one.

Procedure. One script (`run_experiments.sh`) runs every instance × model × configuration with the same limits. It validates each plan with `Validate -v` on the same domain and problem files, and writes one row per run to `results/results.csv`. Plans, VAL outputs and Fast Downward logs are kept in `plans/`, `validation/` and `logs/`. For `lama`, which writes a cheaper plan file each time it improves, the last plan is validated and reported. The columns are measured as follows:

- *Runtime* is Fast Downward's "Planner time" (translation plus search).
- *Expanded states* is the last "Expanded" value in the log.
- *Plan cost* is VAL's final value of `total-cost`.

Name	Search	Heuristic	Optimal	Instances
lama-first	lazy greedy, preferred operators	h^{FF} + landmark count (unit costs)	no	all
lama	lama-first, then weighted A* ($w = 5, 3, 2, 1$)	h^{FF} + landmark count	no	large
astar-lmcut	A*	LM-cut	yes	all
astar-hmax	A*	h^{max}	yes	all
astar-blind	A*	blind	yes	all
gbfs-ff	eager greedy	h^{FF}	no	all

Table 5: Fast Downward configurations. Every run: `-overall-time-limit 5m -overall-memory-limit 4G`.

- An unsolved run is marked as a time or memory failure from the planner's own message.

`lama` keeps searching after each plan. It stops early only when its last weighted A* iteration ($w = 1$) ends without finding a cheaper plan (the single-drone variant, under 1 s). Otherwise it runs until the time limit, so its runtime does not measure how long the best plan took. Its expanded states count only the last iteration. All 66 runs (including the extra instances) were run in one batch on an idle machine, so the runtimes are comparable. 57 runs found a plan and VAL accepted all 57; the other 9 hit a limit (8 time, 1 memory). As a sanity check, we also ran the example of Figure 1 in the statement (`test-problem.pddl`). Every configuration returns cost 10 in both models, as the statement says.

7 Results and Analysis

Table 6 gives every combination of instance, domain model and configuration (statement, Section 7.2). Every plan found was accepted by VAL; the plan cost is VAL's final value of `total-cost`.

Single-drone comparison. Table 7 compares the best validated plan cost of the single-drone and three-drone variants under the same model and configuration (statement, Section 5). The difference is given in absolute terms and as a percentage of the single-drone cost.

Extra experiment: clustered start. Table 8 gives the variant where the three drones all start at the port, with the same map and goals as the large instance.

Unsolved runs. Nine runs did not finish (Table 9). None of them means an instance is unsolvable: every variant has a valid plan in both models from at least one configuration. When A* stops, the largest f value it has reached is a lower bound on the optimal cost, so each unsolved run still tells us how far it got.

Anytime LAMA. Table 10 shows how `lama` improves on its first plan, which is the LAMA-first plan.

7.1 Small instance

No configuration struggled: every run finishes in under 0.35 s with at most 106 expanded states. The battery encoding is tiny (5 levels, one flight cost, four `battery_after` rows), and there is only one drone, so the reachable state space is a few hundred states. The instance was designed to test the encoding, not the planner. Its interest is that the recharge constraint holds: every plan, optimal or not, contains a recharge at `montjuic_pad`, as the argument in Section 5 predicts.

Instance	Model	Planner	Solved	Runtime (s)	Length	Cost	Expanded	VAL
small	A	lama-first	yes	0.34	12	12	21	valid
small	A	astar-lmcut	yes	0.18	10	10	16	valid
small	A	astar-blind	yes	0.16	10	10	73	valid
small	A	astar-hmax	yes	0.19	10	10	37	valid
small	A	gbfs-ff	yes	0.14	12	12	17	valid
small	B	lama-first	yes	0.13	10	10	28	valid
small	B	astar-lmcut	yes	0.14	10	10	16	valid
small	B	astar-blind	yes	0.20	10	10	106	valid
small	B	astar-hmax	yes	0.14	10	10	46	valid
small	B	gbfs-ff	yes	0.14	14	14	20	valid
medium	A	lama-first	yes	0.10	41	52	182	valid
medium	A	astar-lmcut	yes	7.99	24	26	123082	valid
medium	A	astar-blind	yes	1.13	21	26	690566	valid
medium	A	astar-hmax	yes	2.15	22	26	418429	valid
medium	A	gbfs-ff	yes	0.09	27	33	56	valid
medium	B	lama-first	yes	0.12	32	39	6162	valid
medium	B	astar-lmcut	yes	14.17	29	31	336166	valid
medium	B	astar-blind	yes	1.29	27	31	782207	valid
medium	B	astar-hmax	yes	2.66	26	31	547143	valid
medium	B	gbfs-ff	yes	0.10	39	45	143	valid
large	A	lama-first	yes	0.11	44	57	145	valid
large	A	astar-lmcut	yes	58.14	26	30	115435	valid
large	A	lama	yes	294.90*	26	30	328780*	valid
large	A	astar-blind	no (memory)	292.11	–	–	–	–
large	A	astar-hmax	no (time)	292.89	–	–	–	–
large	A	gbfs-ff	yes	0.14	37	44	632	valid
large	B	lama-first	yes	0.15	51	60	3452	valid
large	B	astar-lmcut	yes	122.55	27	31	242786	valid
large	B	lama	yes	292.64*	27	31	26521*	valid
large	B	astar-blind	no (time)	292.62	–	–	–	–
large	B	astar-hmax	no (time)	292.44	–	–	–	–
large	B	gbfs-ff	yes	0.12	39	44	66	valid
large-1drone	A	lama-first	yes	0.11	50	64	141	valid
large-1drone	A	astar-lmcut	yes	0.39	33	38	4456	valid
large-1drone	A	lama	yes	0.72	34	38	67665*	valid
large-1drone	A	astar-blind	yes	0.17	33	38	21639	valid
large-1drone	A	astar-hmax	yes	0.22	33	38	13624	valid
large-1drone	A	gbfs-ff	yes	0.13	44	54	102	valid
large-1drone	B	lama-first	yes	0.18	62	72	2996	valid
large-1drone	B	astar-lmcut	yes	0.83	45	50	11443	valid
large-1drone	B	lama	yes	0.80	45	50	54127*	valid
large-1drone	B	astar-blind	yes	0.15	45	50	25321	valid
large-1drone	B	astar-hmax	yes	0.18	45	50	17832	valid
large-1drone	B	gbfs-ff	yes	0.11	70	82	236	valid

Table 6: Results for every instance, model and configuration (5 min, 4 GB per run). Unsolved runs report the time at which they stopped. *`lama`: runtime until the search stopped, not the time to the best plan; expanded states of the last iteration only.

Model	Planner	1 drone	3 drones	Difference	%
A	astar-lmcut (optimal)	38	30	−8	−21.1
A	lama	38	30	−8	−21.1
A	lama-first	64	57	−7	−10.9
A	gbfs-ff	54	44	−10	−18.5
B	astar-lmcut (optimal)	50	31	−19	−38.0
B	lama	50	31	−19	−38.0
B	lama-first	72	60	−12	−16.7
B	gbfs-ff	82	44	−38	−46.3

Table 7: Plan cost, large instance: one drone vs three drones (blind and h^{max} do not solve the three-drone variant).

Model	Planner	Solved	Runtime (s)	Length	Cost	Expanded
A	lama-first	yes	0.15	42	52	118
A	astar-lmcut	yes	202.86	28	31	420442
A	lama	yes	292.28*	29	32	155*
A	astar-blind	no (time)	292.30	–	–	–
A	astar-hmax	no (time)	292.20	–	–	–
A	gbfs-ff	yes	0.13	35	40	90
B	lama-first	yes	0.13	54	64	112
B	astar-lmcut	no (time)	292.14	–	–	–
B	lama	yes	292.14*	30	33	32870*
B	astar-blind	no (time)	292.10	–	–	–
B	astar-hmax	no (time)	292.02	–	–	–
B	gbfs-ff	yes	0.13	43	49	248

Table 8: Extra variant `problem-large-clustered.pddl`: three drones, all starting at the port. Every plan found is valid. * as in Table 6.

Both models reach the same optimal cost (10) with the same plan length, because the drone only needs one level to finish and one recharge provides it in both models. Model B still expands slightly more states with blind and h^{max} (106 vs 73, 46 vs 37): it can stop at every intermediate level `b1`–`b4`, and those partial-charge states are new states to explore, even though the optimal plan does not use them. The satisficing planners show the price of ignoring costs: LAMA-first returns cost 12 in Model A: it carries the kit east to Ciutadella first and has to fly back across the whole line to the hospital (7 flights instead of 5). `gbfs-ff` returns 14 in Model B. The lesson from the smallest instance is that the recharge model does not change the answer when recharging is a single step, but it already adds states to the search.

7.2 Medium instance

Every configuration solves the medium instance, but the optimal ones begin to work hard: 123k–782k expanded states. A useful surprise is that A* with LM-cut is the *slowest* optimal configuration here (8.0 s in A, 14.2 s in B), although it expands 5.6 times fewer states than blind search, which takes 1.1 s. LM-cut is computed in every state and is expensive. On an instance this size, cheap expansions win over fewer expansions, which is the accuracy/cost trade-off of heuristics seen in Lecture 2. With two drones and seven battery levels each, the battery alone gives $7^2 = 49$ combinations, multiplied by both positions and all the cargo and mission facts. The models now differ clearly. The optimal plans have the same 13 flights, but Model B needs 6 recharges instead of 1, so the plan is 5 actions longer and costs 5 more (31 vs 26). LM-cut expands 2.7 times more states in B, and runtime almost doubles. The heuristics also help less in B: going from h^{max} to

Run	Limit	Expanded	Lower bound	Optimum	Peak memory
large A, blind	memory	65.9M	26	30	4 GiB (limit)
large B, blind	time	66.8M	25	31	3.67 GiB
large A, h^{max}	time	15.7M	27	30	1.57 GiB
large B, h^{max}	time	15.3M	26	31	1.52 GiB
clustered A, blind	time	62.3M	24	31	3.83 GiB
clustered A, h^{max}	time	12.6M	25	31	1.58 GiB
clustered B, blind	time	61.2M	23	31–33	3.79 GiB
clustered B, h^{max}	time	16.4M	25	31–33	1.60 GiB
clustered B, LM-cut	time	345k	31	31–33	0.13 GiB

Table 9: Unsolved runs: states expanded before stopping, highest f reached (a proven lower bound) and the optimal cost. For clustered B the optimum lies between that bound and the best plan found by `lama`.

Variant	Model	First plan	Best plan	Time to best	Optimum
large (3 drones)	A	57	30	9.5 s	30
large (3 drones)	B	60	31	1.3 s	31
large-1drone	A	64	38	0.36 s	38
large-1drone	B	72	50	0.41 s	50
clustered	A	52	32	0.05 s	31
clustered	B	64	33	1.8 s	31–33

Table 10: `lama`: cost of the first and the best plan, and when the best plan was found (from the logs).

LM-cut saves a factor of 3.4 in A but only 1.6 in B. Our explanation is that delete-relaxation heuristics do not see the battery running out (Section 4), so the extra recharges of Model B are a part of the plan that no heuristic estimates. The satisficing results are noisy: LAMA-first is better in B (39) than in A (52), and `gbfs-ff` the other way round (33 vs 45). With costs ignored or poorly estimated, small changes in the model change which plateau the greedy search falls into. The modelling lesson is that a more faithful recharge does not make the task bigger, but it does make the plans deeper and the heuristic weaker.

7.3 Large instance

This is where the planners struggle. A* with LM-cut is the only optimal configuration that solves the three-drone instance (58 s in A, 123 s in B). Blind search and h^{max} expand 15–67 million states without finishing, and they are still far from the optimum when they stop (lower bounds 25–27 against 30–31). Blind search hits the memory limit in Model A: with three drones every level of the search multiplies the frontier, which is the exponential growth of uninformed search. The difficulty comes mostly from the number of drones, not from the size of the map. The single-drone variant has the same 12 locations and goals but only 196 grounded actions instead of 588, and LM-cut solves it in 0.4 s, even though its optimal plan is longer (33 vs 26 actions). Each drone adds a position, a battery level (7 values, with `battery_after` rows for costs 1, 2 and 3) and a cargo slot to every state, so the state space grows as a power of the number of drones. With three drones Model B costs only one unit more than A (31 vs 30, one recharge each), because every drone can work close to a charger. Its search is still twice as large (243k vs 115k expansions). The satisficing results show why the statement asks for `lama`: LAMA-first's plans cost almost twice the optimum (57 and 60). It ignores action costs (`cost_type=one`), so a cost-3 river crossing looks as cheap as a cost-1 hop. `gbfs-ff` uses the real costs in h^{FF} and does better (44). The anytime `lama` reaches the optimal cost in both models (Table 10), in 9.5 s for A

and 1.3 s for B, which is much faster than A* but without a proof of optimality. The workload in the optimal Model A plan is uneven: `droneB` does 15 actions, `droneA` 9 and `droneC` only 2 (one flight and one inspection). The metric minimises total cost, not mission time, so nothing rewards sharing the work (Section 9).

7.4 Single-drone comparison and clustered start

Three drones reduce the optimal cost by 8 (21.1%) in Model A and by 19 (38.0%) in Model B (Table 7). We compare cost, not length: the classical plan is sequential, so its length counts every drone's actions. The optimal single-drone plans have the same 17 flights in both models, but A needs 4 recharges and B 16. A lone drone carries one supply at a time, so it crosses the city several times between both depots and the sites, and it must top up its battery repeatedly. In Model B each of those top-ups is paid level by level, which is why the gain from extra drones is almost twice as large there. The three-drone plan needs only one recharge, because each drone starts full in its own region. The greedy configurations show the same direction (11–46%), but their gaps mix the effect of the drones with the quality of the plan, so we base the comparison on the optimal and `lama` costs.

The statement's hint suggests that starting the drones in different regions produces the advantage. We tested this with the clustered variant, where all three drones start at the port. In Model A, going from one drone to three clustered drones already saves 7 of the 8 units (38 → 31), and spreading them saves only 1 more (31 → 30). In Model B, clustering saves between 17 and 19 of the 19 units (the optimum is between 31 and 33). On our map, most of the gain comes from having more batteries and cargo slots, not from where the drones start. Our explanation is that the centre of the map (`estacio`, `rambla`) keeps every region within a few levels of both depots, so a clustered drone loses little by flying to its region first. A map with longer distances between the regions should make the start positions matter more. The clustered variant is also the hardest for the planner: LM-cut needs 420k expansions in A and does not finish in B, against 115k and 243k for the spread start. Three identical drones at the same location create many symmetric states ("A does x and B does y" and the reverse), and A* explores them separately.

7.5 Across instances

- **Heuristic strength.** Wherever blind, h^{max} and LM-cut all finish, they return the same optimal cost and expand fewer states in that order (single drone, Model A: 21,639 → 13,624 → 4,456). This reproduces the dominance idea of Lecture 2 on our own domain. More informed is not always faster, though: on the medium instance the cheaper heuristics finish first.
- **Battery and the delete relaxation.** The gain from h^{max} to LM-cut is about 3 times in Model A and only about 1.6 times in Model B, on both the medium (3.4 vs 1.6) and the single-drone instance (3.1 vs 1.6). This supports our explanation that relaxed heuristics do not count recharges. It is evidence from two instances, not a proof.
- **Cost-aware greedy search.** Over the 10 instance and model pairs (including the clustered variant), `gbfs-ff` finds the cheaper plan in 6, LAMA-first in 3 (small B, medium B, single-drone B), and one is a tie. On the large variants, where the edge costs vary most, `gbfs-ff` wins every case except single-drone Model B. This is in line with LAMA-first ignoring action costs; when many recharges are needed, LAMA's landmarks seem to compensate. Both finish in under 0.4 s, and both are far from optimal: fast planners pay with plan quality, optimal planners pay with time.
- **Grounding is not the bottleneck.** The translator produces at most 588 actions and 103 facts, the same for both models. All the difficulty is in the search: plan depth (recharge model) and the number of drones.

- **Measurement.** Each configuration was run once, so small runtime differences (below about 0.1 s) are not meaningful. Expanded states are deterministic and are the better measure of effort.

8 Literature Connection

Bernardini, Fox and Long (2014), quadcopter surveillance [5]. This paper plans the missions of a real low-cost quadcopter in PDDL, and names its short battery life as the main limit on what the drone can do. The battery is a numeric fluent: each manoeuvre requires `(>= (batteryLevel) ...)` and applies a `decrease`, with the energy of each manoeuvre measured in flight tests. Fast Downward cannot handle numeric fluents, so we compiled the same idea into level objects and a `battery_after` table with one row per valid (level, cost) pair, which a numeric model gets for free. Their second domain adds a `recharge` action that charges "to full capacity or up to a chosen level", the same choice we split into Model A (full) and Model B (one level per action), and close to the variable recharge we propose in Section 9. They also use durative actions with the anytime planner OPTIC, which supports both our anytime `-alias lama` runs on the large instance and our point that only a temporal model can reward drones flying in parallel.

Nakhost, Hoffmann and Müller (2012), resource-constrained planning [6]. This paper studies planning when the resource supply is close to the minimum a plan needs. Its main finding is that planners guided by delete-relaxation heuristics get much worse as resources get tighter, because relaxed plans act as if a resource value, once reached, persists forever. This explains our observation that heuristics guide Model B poorly. Whether the battery is a number or a set of level facts, the relaxed problem never runs out of energy, so recharges stay invisible to the heuristic. In their study, LM-cut worked best with the propositional encoding (resource levels as facts) and LAMA with that encoding plus action costs, which supports our choice to compile the battery into levels for Fast Downward. The connection is partial: their resources cannot be replenished, while our drones can recharge, so our instances are only tight between chargers.

9 Limitations and Future Work

- **No time or concurrency.** Plans are sequential and the metric is the total cost, so the model cannot tell a mission where three drones fly in parallel from one where they take turns. This is why the optimal large plan gives `droneC` only two actions. Durative actions (PDDL 2.1) with a makespan metric, solved by a temporal planner, would reward sharing the work, which matters in an emergency.
- **Simplified energy.** Flight cost depends only on the edge, not on the payload, the wind or the drone. The battery is discretised into a few levels, so small differences in consumption are rounded. A numeric planner (Section 8) could use exact values and payload-dependent consumption.
- **Recharge cost and chargers.** In both models a recharge action costs 1, which in Model A undervalues a full charge. Chargers have unlimited capacity, so several drones can charge at the same place at once. A third model could let the drone choose how many levels to charge in one action, at a cost proportional to the levels, which combines the realism of B with the short plans of A.
- **Capacity and supplies.** A drone carries one supply, and each supply object has one fixed destination. Larger payloads would need a capacity counter, encoded symbolically like the battery.

- **Full knowledge and no failures.** Every lock, cost and need is known in advance, and actions always succeed. Real emergencies involve unknown blockages, new requests and failing drones. Online replanning, or planning under uncertainty (contingent or probabilistic planning), would handle them.
- **Search issues found in the experiments.** Identical drones create symmetric states (clustered variant), and relaxed heuristics ignore battery use. Symmetry reduction, or heuristics that reason about resources, could make the larger instances easier.
- **Experiment scope.** Each run was performed once, and the claims about start positions and battery-blind heuristics rest on a few instances. Repeated runs and a family of generated maps (more drones, longer distances, more battery levels) would make the conclusions stronger.

Use of AI Tools

TODO

References

[1] M. Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

[2] R. Howey, D. Long and M. Fox. VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *Proc. ICTAI*, pages 294–301, 2004.

[3] M. Helmert and C. Domshlak. Landmarks, critical paths and abstractions: What's the difference anyway? In *Proc. ICAPS*, pages 162–169, 2009.

[4] S. Richter and M. Westphal. The LAMA planner: Guiding cost-based anytime planning with landmarks. *Journal of Artificial Intelligence Research*, 39:127–177, 2010.

[5] S. Bernardini, M. Fox and D. Long. Planning the behaviour of low-cost quadcopters for surveillance missions. In *Proc. ICAPS*, pages 445–453, 2014.

[6] H. Nakhost, J. Hoffmann and M. Müller. Resource-constrained planning: A Monte Carlo random walk approach. In *Proc. ICAPS*, pages 181–189, 2012.

A Appendix

All files are included exactly as they are given to the planner. The extra instances (`test-problem.pddl`, `optional_experiments/problem-large-clustered.pddl`), every plan, every VAL output and every log are in the submitted folder.

A.1 Domain files

`domain-A.pddl`

```
;; ============================================================================
;; domain-A.pddl : Autonomous Drone Planning for Smart-City Emergency Response
;; MODEL A : one recharge action sets the battery directly to the MAXIMUM level
;; (whatever the current level is).
;; Planner : Fast Downward (no numeric fluents; battery is symbolic via levels)
;; ============================================================================

(define (domain drone_emergency) ; same name in both models: shared problem files
```

```
;; ------------------------------------------------------------------------
;; Requirements
;; No :negative-preconditions: every "not yet" condition is a positive fact
;; (cargo_empty, needs_inspection, next_level).
;; ------------------------------------------------------------------------
(:requirements
  :strips ; basic add/delete effects
  :typing ; typed objects
  :conditional-effects ; forall/when used to unlock locations
  :action-costs ; total-cost as plan cost
)

;; ------------------------------------------------------------------------
;; Types (NOTE: "- object" on the first line, otherwise they become subtypes
;; of supply)
;; ------------------------------------------------------------------------
(:types
  drone location supply level - object ; the four mandatory base types
  medical_kit water_package communication_device - supply ; kinds of supply
)

;; ------------------------------------------------------------------------
;; Functions (action costs only; no numeric fluents are used by the planner
;; in conditions or effects other than total-cost)
;; ------------------------------------------------------------------------
(:functions
  (total-cost) - number ; accumulated plan cost
  (distance ?from_location ?to_location - location) - number ; static flight distance (= flight cost)
)

;; ------------------------------------------------------------------------
;; Predicates
;; ------------------------------------------------------------------------
(:predicates
  ;; ---- drones ----
  (drone_at ?drone - drone ?location - location) ; drone is currently at this location
  (carrying ?drone - drone ?supply - supply) ; drone is carrying this supply
  (cargo_empty ?drone - drone) ; drone carries nothing (capacity = 1 supply)

  ;; ---- map ----
  (connected ?from_location ?to_location - location) ; drone can fly directly between the two (both
      directions are declared)
  (has_charging_station ?location - location) ; location has a charging station
  (accessible ?location - location) ; location can currently be entered (not locked)
  (unlocked_by_inspection ?inspected_location ?locked_location - location) ; inspecting the first
      location unlocks the second
  (unlocked_by_delivery ?supply - supply ?locked_location - location) ; delivering this supply unlocks
      the location

  ;; ---- mission ----
  (needs_inspection ?location - location) ; mission site not inspected yet (deleted by inspect)
  (inspected ?location - location) ; location has been inspected (goal predicate)
  (needs_supply ?location - location ?supply - supply) ; location is waiting for this specific supply
  (supply_at ?supply - supply ?location - location) ; supply is lying at this location (not carried)
  (delivered ?supply - supply ?location - location) ; supply has been delivered to this location

  ;; ---- symbolic battery ----
  (battery ?drone - drone ?level - level) ; current battery level of the drone
  (flight_cost ?from_location ?to_location - location ?cost - level) ; flying from->to costs this many
      levels
  (battery_after ?before - level ?cost - level ?left - level) ; before - cost = left (only listed if
      before >= cost)
  (next_level ?lower_level ?higher_level - level) ; higher = lower + 1 (Model A: only as "battery not
      full" check)
  (max_level ?level - level) ; full battery capacity level
)

;; ------------------------------------------------------------------------
;; ACTION fly : move a drone to a connected, accessible location
;; consumes battery equal to the flight cost
;; ------------------------------------------------------------------------
(:action fly
```

```
  :parameters (?drone - drone ?from_location ?to_location - location
               ?battery_before ?cost ?battery_left - level)
  :precondition (and
    (drone_at ?drone ?from_location) ; drone starts at ?from_location
    (connected ?from_location ?to_location) ; the two locations are connected
    (accessible ?to_location) ; destination is not locked
    (battery ?drone ?battery_before) ; current battery level
    (flight_cost ?from_location ?to_location ?cost) ; cost of this flight in levels
    (battery_after ?battery_before ?cost ?battery_left) ; exists only if enough battery
  )
  :effect (and
    (not (drone_at ?drone ?from_location)) ; leave origin
    (drone_at ?drone ?to_location) ; arrive at destination
    (not (battery ?drone ?battery_before)) ; remove old battery level
    (battery ?drone ?battery_left) ; set new battery level
    (increase (total-cost) (distance ?from_location ?to_location)) ; plan cost = distance
  )
)

;; ------------------------------------------------------------------------
;; ACTION inspect : inspect the location where the drone is
;; may unlock other locations
;; ------------------------------------------------------------------------
(:action inspect
  :parameters (?drone - drone ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is present at the location
    (needs_inspection ?location) ; only mission sites, and only once
  )
  :effect (and
    (inspected ?location) ; location is now inspected
    (not (needs_inspection ?location)) ; blocks a second inspection
    (forall (?locked_location - location) ; unlock every location depending on this inspection
      (when (unlocked_by_inspection ?location ?locked_location)
        (accessible ?locked_location)))
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ------------------------------------------------------------------------
;; ACTION pick-up-supply : drone picks up a supply at its own location
;; ------------------------------------------------------------------------
(:action pick-up-supply
  :parameters (?drone - drone ?supply - supply ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the location
    (supply_at ?supply ?location) ; supply is at the same location
    (cargo_empty ?drone) ; drone is not already carrying a supply
  )
  :effect (and
    (not (supply_at ?supply ?location)) ; supply leaves the location
    (carrying ?drone ?supply) ; drone now carries it
    (not (cargo_empty ?drone)) ; drone cargo is occupied
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ------------------------------------------------------------------------
;; ACTION deliver-supply : deliver the carried supply to a location needing it
;; may unlock other locations
;; ------------------------------------------------------------------------
(:action deliver-supply
  :parameters (?drone - drone ?supply - supply ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the target location
    (carrying ?drone ?supply) ; drone carries the supply
    (needs_supply ?location ?supply) ; this location needs exactly this supply
  )
  :effect (and
    (not (carrying ?drone ?supply)) ; drone no longer carries it
    (cargo_empty ?drone) ; cargo is free again
    (not (needs_supply ?location ?supply)) ; need is satisfied
    (delivered ?supply ?location) ; record the delivery (goal predicate)
```

```
    (forall (?locked_location - location) ; unlock locations depending on this delivery
      (when (unlocked_by_delivery ?supply ?locked_location)
        (accessible ?locked_location)))
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ---------------------------------------------------------------------------
;; ACTION recharge (MODEL A) : at a charging station, jump straight to the
;; maximum battery level, whatever the current one
;; ---------------------------------------------------------------------------
(:action recharge
  :parameters (?drone - drone ?location - location
               ?current_level ?higher_level ?full_level - level)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the location
    (has_charging_station ?location) ; location has a charging station
    (battery ?drone ?current_level) ; current battery level
    (next_level ?current_level ?higher_level) ; battery not already full (no next_level above the max);
                                                ; ?higher_level is only this check, not the new
                                                    level
    (max_level ?full_level) ; the full-capacity level object
  )
  :effect (and
    (not (battery ?drone ?current_level)) ; remove old level
    (battery ?drone ?full_level) ; battery set directly to full
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

)
```

domain-B.pddl

```
;; ===========================================================================
;; domain-B.pddl : Autonomous Drone Planning for Smart-City Emergency Response
;; MODEL B : each recharge action raises the battery by ONE level (next_level);
;; several recharge actions are needed to reach full capacity.
;; Planner : Fast Downward (no numeric fluents; battery is symbolic via levels)
;; Only the recharge action differs from domain-A.pddl.
;; ===========================================================================

(define (domain drone_emergency) ; same name in both models: shared problem files

  ;; ---------------------------------------------------------------------------
  ;; Requirements
  ;; No :negative-preconditions: every "not yet" condition is a positive fact
  ;; (cargo_empty, needs_inspection, next_level).
  ;; ---------------------------------------------------------------------------
  (:requirements
    :strips ; basic add/delete effects
    :typing ; typed objects
    :conditional-effects ; forall/when used to unlock locations
    :action-costs ; total-cost as plan cost
  )

  ;; ---------------------------------------------------------------------------
  ;; Types (NOTE: "- object" on the first line, otherwise they become subtypes
  ;; of supply)
  ;; ---------------------------------------------------------------------------
  (:types
    drone location supply level - object ; the four mandatory base types
    medical_kit water_package communication_device - supply ; kinds of supply
  )

  ;; ---------------------------------------------------------------------------
  ;; Functions
  ;; ---------------------------------------------------------------------------
  (:functions
    (total-cost) - number ; accumulated plan cost
    (distance ?from_location ?to_location - location) - number ; static flight distance (= flight cost)
  )
```

```
;; ------------------------------------------------------------------------
;; Predicates
;; ------------------------------------------------------------------------
(:predicates
  ;; ---- drones ----
  (drone_at ?drone - drone ?location - location) ; drone is currently at this location
  (carrying ?drone - drone ?supply - supply) ; drone is carrying this supply
  (cargo_empty ?drone - drone) ; drone carries nothing (capacity = 1 supply)

  ;; ---- map ----
  (connected ?from_location ?to_location - location) ; drone can fly directly between the two (both
      directions are declared)
  (has_charging_station ?location - location) ; location has a charging station
  (accessible ?location - location) ; location can currently be entered (not locked)
  (unlocked_by_inspection ?inspected_location ?locked_location - location) ; inspecting the first
      location unlocks the second
  (unlocked_by_delivery ?supply - supply ?locked_location - location) ; delivering this supply unlocks
      the location

  ;; ---- mission ----
  (needs_inspection ?location - location) ; mission site not inspected yet (deleted by inspect)
  (inspected ?location - location) ; location has been inspected (goal predicate)
  (needs_supply ?location - location ?supply - supply) ; location is waiting for this specific supply
  (supply_at ?supply - supply ?location - location) ; supply is lying at this location (not carried)
  (delivered ?supply - supply ?location - location) ; supply has been delivered to this location

  ;; ---- symbolic battery ----
  (battery ?drone - drone ?level - level) ; current battery level of the drone
  (flight_cost ?from_location ?to_location - location ?cost - level) ; flying from->to costs this many
      levels
  (battery_after ?before - level ?cost - level ?left - level) ; before - cost = left (only listed if
      before >= cost)
  (next_level ?lower_level ?higher_level - level) ; higher = lower + 1 (used by partial recharge)
  (max_level ?level - level) ; full battery capacity level (declared for same problem files as Model A)
)

;; ------------------------------------------------------------------------
;; ACTION fly : move a drone to a connected, accessible location
;; consumes battery equal to the flight cost
;; ------------------------------------------------------------------------
(:action fly
  :parameters (?drone - drone ?from_location ?to_location - location
               ?battery_before ?cost ?battery_left - level)
  :precondition (and
    (drone_at ?drone ?from_location) ; drone starts at ?from_location
    (connected ?from_location ?to_location) ; the two locations are connected
    (accessible ?to_location) ; destination is not locked
    (battery ?drone ?battery_before) ; current battery level
    (flight_cost ?from_location ?to_location ?cost) ; cost of this flight in levels
    (battery_after ?battery_before ?cost ?battery_left) ; exists only if enough battery
  )
  :effect (and
    (not (drone_at ?drone ?from_location)) ; leave origin
    (drone_at ?drone ?to_location) ; arrive at destination
    (not (battery ?drone ?battery_before)) ; remove old battery level
    (battery ?drone ?battery_left) ; set new battery level
    (increase (total-cost) (distance ?from_location ?to_location)) ; plan cost = distance
  )
)

;; ------------------------------------------------------------------------
;; ACTION inspect : inspect the location where the drone is
;; may unlock other locations
;; ------------------------------------------------------------------------
(:action inspect
  :parameters (?drone - drone ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is present at the location
    (needs_inspection ?location) ; only mission sites, and only once
  )
  :effect (and
    (inspected ?location) ; location is now inspected
    (not (needs_inspection ?location)) ; blocks a second inspection
```

```
    (forall (?locked_location - location) ; unlock every location depending on this inspection
      (when (unlocked_by_inspection ?location ?locked_location)
        (accessible ?locked_location)))
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ------------------------------------------------------------------------
;; ACTION pick-up-supply : drone picks up a supply at its own location
;; ------------------------------------------------------------------------
(:action pick-up-supply
  :parameters (?drone - drone ?supply - supply ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the location
    (supply_at ?supply ?location) ; supply is at the same location
    (cargo_empty ?drone) ; drone is not already carrying a supply
  )
  :effect (and
    (not (supply_at ?supply ?location)) ; supply leaves the location
    (carrying ?drone ?supply) ; drone now carries it
    (not (cargo_empty ?drone)) ; drone cargo is occupied
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ------------------------------------------------------------------------
;; ACTION deliver-supply : deliver the carried supply to a location needing it
;; may unlock other locations
;; ------------------------------------------------------------------------
(:action deliver-supply
  :parameters (?drone - drone ?supply - supply ?location - location)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the target location
    (carrying ?drone ?supply) ; drone carries the supply
    (needs_supply ?location ?supply) ; this location needs exactly this supply
  )
  :effect (and
    (not (carrying ?drone ?supply)) ; drone no longer carries it
    (cargo_empty ?drone) ; cargo is free again
    (not (needs_supply ?location ?supply)) ; need is satisfied
    (delivered ?supply ?location) ; record the delivery (goal predicate)
    (forall (?locked_location - location) ; unlock locations depending on this delivery
      (when (unlocked_by_delivery ?supply ?locked_location)
        (accessible ?locked_location)))
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

;; ------------------------------------------------------------------------
;; ACTION recharge (MODEL B) : at a charging station, raise the battery by
;; exactly one level per action
;; ------------------------------------------------------------------------
(:action recharge
  :parameters (?drone - drone ?location - location
               ?current_level ?higher_level - level)
  :precondition (and
    (drone_at ?drone ?location) ; drone is at the location
    (has_charging_station ?location) ; location has a charging station
    (battery ?drone ?current_level) ; current battery level
    (next_level ?current_level ?higher_level) ; higher = current + 1 (does not exist for the max level,
         so no overcharge)
  )
  :effect (and
    (not (battery ?drone ?current_level)) ; remove old level
    (battery ?drone ?higher_level) ; battery raised by one level
    (increase (total-cost) 1) ; every non-flight action costs 1
  )
)

)
```

A.2 Problem files

problem-small.pddl

```
;; ============================================================================
;; problem-small.pddl : Barcelona Flood Emergency - Small Instance
;;
;; Scenario: A DANA storm brings torrential rain and a Mediterranean storm surge,
;; flooding Barcelona's coastal Barceloneta district and cutting off access to
;; Parc de la Ciutadella, which is being used as an evacuation centre. The
;; flooded Llobregat cuts the road to Hospital de Viladecans, south-west of
;; El Prat. One drone (droneA) is dispatched from El Prat emergency
;; aerodrome, the central depot where the medical kit is stored.
;;
;; Objects:
;; 1 drone : droneA (starts at aerodrome, full battery b4)
;; 5 locations, a line from south-west to north-east:
;; hospital_viladecans - aerodrome - barceloneta - montjuic_pad - ciutadella
;; hospital_viladecans - Hospital de Viladecans, across the Llobregat (deliver kit1)
;; aerodrome - El Prat emergency drone base (DEPOT, kit1 here, no charger)
;; barceloneta - flooded beach district (inspect -> unlocks ciutadella)
;; montjuic_pad - Montjuic hilltop charging pad (CHARGING STATION)
;; ciutadella - Parc de la Ciutadella hub (LOCKED, inspect)
;; 1 supply: kit1 (medical_kit) at the depot (aerodrome)
;;
;; Design constraint (mandatory recharge):
;; droneA starts FULL (b4), so the map itself forces the recharge.
;; The map is a line and the drone starts between its two ends:
;; hospital_viladecans (one side) and ciutadella (other side) must both be visited.
;; So the drone must reach one end and come back: the cheapest way is
;; aerodrome->hospital_viladecans->aerodrome (2 flights), then
;; aerodrome->barceloneta->montjuic_pad->ciutadella (3 flights).
;; Every complete mission needs at least 5 flights = 5 levels > 4.
;; No recharge-free solution exists. On the best route the drone reaches
;; montjuic_pad (the only charger) with b0, just before the last flight.
;;
;; Optimal plan cost: 10 in both models
;; (5 flights + 2 inspections + pick-up + delivery + 1 recharge)
;; Model A (instant full recharge): 10
;; Model B (incremental +1/action): 10 (one recharge is enough: b0 -> b1
;; covers the last flight montjuic_pad->ciutadella)
;; ============================================================================

(define (problem barcelona-flood-small)
  (:domain drone_emergency)

  (:objects
    droneA - drone
    aerodrome montjuic_pad barceloneta hospital_viladecans ciutadella - location
    kit1 - medical_kit
    b0 b1 b2 b3 b4 - level
  )

  (:init

    ;; ---- drone ------------------------------------------------------------
    (drone_at droneA aerodrome) ; droneA at El Prat emergency base
    (battery droneA b4) ; full battery: the map, not the start, forces the recharge
    (cargo_empty droneA)

    ;; ---- map: connections (bidirectional) ---------------------------------
    ;; Line: hospital_viladecans - aerodrome - barceloneta - montjuic_pad - ciutadella
    ;; Both ends are mission sites, so the drone must go to one end and back:
    ;; this is what makes every complete route at least 5 flights long
    (connected aerodrome hospital_viladecans) (connected hospital_viladecans aerodrome)
    (connected aerodrome barceloneta) (connected barceloneta aerodrome)
    (connected barceloneta montjuic_pad) (connected montjuic_pad barceloneta)
    (connected montjuic_pad ciutadella) (connected ciutadella montjuic_pad)

    ;; ---- flight costs (all 1 level per edge) ------------------------------
    (flight_cost aerodrome hospital_viladecans b1) (flight_cost hospital_viladecans aerodrome b1)
    (flight_cost aerodrome barceloneta b1) (flight_cost barceloneta aerodrome b1)
    (flight_cost barceloneta montjuic_pad b1) (flight_cost montjuic_pad barceloneta b1)
```

```
    (flight_cost montjuic_pad ciutadella b1) (flight_cost ciutadella montjuic_pad b1)

    ;; ---- distances (plan-cost per edge, equals flight cost here) ------------
    (= (distance aerodrome hospital_viladecans) 1) (= (distance hospital_viladecans aerodrome) 1)
    (= (distance aerodrome barceloneta) 1) (= (distance barceloneta aerodrome) 1)
    (= (distance barceloneta montjuic_pad) 1) (= (distance montjuic_pad barceloneta) 1)
    (= (distance montjuic_pad ciutadella) 1) (= (distance ciutadella montjuic_pad) 1)

    ;; ---- infrastructure -----------------------------------------------------
    (has_charging_station montjuic_pad) ; Montjuic hilltop solar charging station

    ;; ---- accessibility ------------------------------------------------------
    ;; ciutadella sealed by flood debris + emergency cordon until Barceloneta
    ;; is inspected and declared passable (safe passage from beach confirmed).
    (accessible aerodrome)
    (accessible montjuic_pad)
    (accessible barceloneta)
    (accessible hospital_viladecans)
    ;; ciutadella NOT listed -- implicitly locked
    (unlocked_by_inspection barceloneta ciutadella)

    ;; ---- supplies -----------------------------------------------------------
    (supply_at kit1 aerodrome) ; medical kit stored at the depot
    (needs_supply hospital_viladecans kit1) ; Hospital de Viladecans awaits the kit

    ;; ---- mission requirements -----------------------------------------------
    (needs_inspection barceloneta) ; structural damage / flood depth survey
    (needs_inspection ciutadella) ; evacuation-centre capacity assessment

    ;; ---- symbolic battery arithmetic (capacity b4 = 4 levels) ---------------
    ;; battery_after ?before ?cost ?result (before - cost = result, only if >= 0)
    (battery_after b4 b1 b3) (battery_after b3 b1 b2)
    (battery_after b2 b1 b1) (battery_after b1 b1 b0)

    ;; next_level: used by Model B incremental recharge & Model A not-full check
    (next_level b0 b1) (next_level b1 b2) (next_level b2 b3) (next_level b3 b4)
    (max_level b4) ; Model A: one recharge action jumps directly to b4

    (= (total-cost) 0)
  )

  (:goal (and
    (inspected barceloneta) ; Barceloneta district cleared for ground teams
    (inspected ciutadella) ; Parc de la Ciutadella evacuation hub assessed
    (delivered kit1 hospital_viladecans) ; Hospital de Viladecans receives emergency supplies
  ))

  (:metric minimize (total-cost))
)
```

problem-medium.pddl

```
;; ============================================================================
;; problem-medium.pddl : Barcelona Flood Emergency - Medium Instance
;;
;; Scenario: Severe storm surge and flash flooding strike Barcelona along the
;; coastal strip and the Besos river basin. Two drones are coordinated:
;; - droneA starts at Port Vell Emergency Depot (port_depot) with battery b5.
;; - droneB starts at Torre Glories Tech District Charging Hub (glories_tower) with battery b4.
;;
;; Objects:
;; 2 drones: droneA, droneB
;; 8 locations:
;; 1 depot:
;; port_depot - Port Vell emergency logistics center (stores all supplies)
;; 2 charging stations:
;; montjuic_pad - Montjuic hilltop telecommunications & solar charging pad
;; glories_tower - Torre Glories charging hub / electrical substation
;; 5 mission sites:
;; barceloneta - Flooded beachside district (needs inspection)
;; hospital_mar - Coastal hospital cut off by floodwaters (needs kit1 & inspection)
;; poblenou - Industrial & residential district with flooded sewers (needs water1 & inspection)
;; forum_shelter - Parc del Forum coastal evacuation center (needs comm1)
```

```
;; besos_delta - Besos river delta & flood barrier (LOCKED until poblenou inspected; needs inspection)
;;
;; 3 supplies (3 different types):
;; kit1 - medical_kit (staged at port_depot, needed at hospital_mar)
;; water1 - water_package (staged at port_depot, needed at poblenou)
;; comm1 - communication_device (staged at port_depot, needed at forum_shelter)
;;
;; Design constraint (locking precondition):
;; besos_delta cannot be entered initially because (accessible besos_delta) is false.
;; Inspecting poblenou unlocks besos_delta via (unlocked_by_inspection poblenou besos_delta).
;; Any action sequence attempting to fly directly to besos_delta beforehand (e.g.
;; from glories_tower, poblenou, or forum_shelter) fails because the (accessible besos_delta)
;; precondition in the 'fly' action is not satisfied.
;; ============================================================================

(define (problem barcelona-flood-medium)
  (:domain drone_emergency)

  (:objects
    droneA droneB - drone
    port_depot montjuic_pad glories_tower barceloneta hospital_mar poblenou forum_shelter besos_delta -
        location
    kit1 - medical_kit
    water1 - water_package
    comm1 - communication_device
    b0 b1 b2 b3 b4 b5 b6 - level
  )

  (:init

    ;; ---- drones ------------------------------------------------------------
    (drone_at droneA port_depot)
    (battery droneA b5)
    (cargo_empty droneA)

    (drone_at droneB glories_tower)
    (battery droneB b4)
    (cargo_empty droneB)

    ;; ---- map: connections (bidirectional) ----------------------------------
    (connected port_depot montjuic_pad) (connected montjuic_pad port_depot)
    (connected port_depot barceloneta) (connected barceloneta port_depot)
    (connected port_depot glories_tower) (connected glories_tower port_depot)
    (connected montjuic_pad barceloneta) (connected barceloneta montjuic_pad)
    (connected barceloneta hospital_mar) (connected hospital_mar barceloneta)
    (connected hospital_mar glories_tower) (connected glories_tower hospital_mar)
    (connected hospital_mar poblenou) (connected poblenou hospital_mar)
    (connected glories_tower poblenou) (connected poblenou glories_tower)
    (connected glories_tower besos_delta) (connected besos_delta glories_tower)
    (connected poblenou forum_shelter) (connected forum_shelter poblenou)
    (connected poblenou besos_delta) (connected besos_delta poblenou)
    (connected forum_shelter besos_delta) (connected besos_delta forum_shelter)

    ;; ---- flight costs (varying 1 and 2 levels per edge) --------------------
    ;; Cost 1 edges:
    (flight_cost port_depot montjuic_pad b1) (flight_cost montjuic_pad port_depot b1)
    (flight_cost port_depot barceloneta b1) (flight_cost barceloneta port_depot b1)
    (flight_cost barceloneta hospital_mar b1) (flight_cost hospital_mar barceloneta b1)
    (flight_cost hospital_mar poblenou b1) (flight_cost poblenou hospital_mar b1)
    (flight_cost glories_tower poblenou b1) (flight_cost poblenou glories_tower b1)
    (flight_cost poblenou forum_shelter b1) (flight_cost forum_shelter poblenou b1)
    (flight_cost poblenou besos_delta b1) (flight_cost besos_delta poblenou b1)
    (flight_cost forum_shelter besos_delta b1) (flight_cost besos_delta forum_shelter b1)

    ;; Cost 2 edges:
    (flight_cost port_depot glories_tower b2) (flight_cost glories_tower port_depot b2)
    (flight_cost montjuic_pad barceloneta b2) (flight_cost barceloneta montjuic_pad b2)
    (flight_cost hospital_mar glories_tower b2) (flight_cost glories_tower hospital_mar b2)
    (flight_cost glories_tower besos_delta b2) (flight_cost besos_delta glories_tower b2)

    ;; ---- distances (metric plan-cost per edge) -----------------------------
    (= (distance port_depot montjuic_pad) 1) (= (distance montjuic_pad port_depot) 1)
    (= (distance port_depot barceloneta) 1) (= (distance barceloneta port_depot) 1)
```

```
  (= (distance barceloneta hospital_mar) 1) (= (distance hospital_mar barceloneta) 1)
  (= (distance hospital_mar poblenou) 1) (= (distance poblenou hospital_mar) 1)
  (= (distance glories_tower poblenou) 1) (= (distance poblenou glories_tower) 1)
  (= (distance poblenou forum_shelter) 1) (= (distance forum_shelter poblenou) 1)
  (= (distance poblenou besos_delta) 1) (= (distance besos_delta poblenou) 1)
  (= (distance forum_shelter besos_delta) 1) (= (distance besos_delta forum_shelter) 1)

  (= (distance port_depot glories_tower) 2) (= (distance glories_tower port_depot) 2)
  (= (distance montjuic_pad barceloneta) 2) (= (distance barceloneta montjuic_pad) 2)
  (= (distance hospital_mar glories_tower) 2) (= (distance glories_tower hospital_mar) 2)
  (= (distance glories_tower besos_delta) 2) (= (distance besos_delta glories_tower) 2)

  ;; ---- infrastructure ------------------------------------------------------
  (has_charging_station montjuic_pad)
  (has_charging_station glories_tower)

  ;; ---- accessibility -------------------------------------------------------
  (accessible port_depot)
  (accessible montjuic_pad)
  (accessible glories_tower)
  (accessible barceloneta)
  (accessible hospital_mar)
  (accessible poblenou)
  (accessible forum_shelter)
  ;; besos_delta is initially locked until poblenou is inspected
  (unlocked_by_inspection poblenou besos_delta)

  ;; ---- supplies (all 3 initially staged at port_depot) ---------------------
  (supply_at kit1 port_depot)
  (supply_at water1 port_depot)
  (supply_at comm1 port_depot)

  ;; ---- delivery requirements (3 deliveries of 3 different types) -----------
  (needs_supply hospital_mar kit1)
  (needs_supply poblenou water1)
  (needs_supply forum_shelter comm1)

  ;; ---- inspection requirements (4 inspections) -----------------------------
  (needs_inspection barceloneta)
  (needs_inspection hospital_mar)
  (needs_inspection poblenou)
  (needs_inspection besos_delta)

  ;; ---- symbolic battery arithmetic (capacity b6 = 6 levels) ----------------
  ;; battery_after for cost b1:
  (battery_after b6 b1 b5) (battery_after b5 b1 b4)
  (battery_after b4 b1 b3) (battery_after b3 b1 b2)
  (battery_after b2 b1 b1) (battery_after b1 b1 b0)

  ;; battery_after for cost b2:
  (battery_after b6 b2 b4) (battery_after b5 b2 b3)
  (battery_after b4 b2 b2) (battery_after b3 b2 b1)
  (battery_after b2 b2 b0)

  ;; next_level chain (b0 to b6):
  (next_level b0 b1) (next_level b1 b2) (next_level b2 b3)
  (next_level b3 b4) (next_level b4 b5) (next_level b5 b6)
  (max_level b6)

  (= (total-cost) 0)
)

(:goal (and
  ;; 4 inspection goals:
  (inspected barceloneta)
  (inspected hospital_mar)
  (inspected poblenou)
  (inspected besos_delta)

  ;; 3 delivery goals (3 distinct supply types):
  (delivered kit1 hospital_mar)
  (delivered water1 poblenou)
  (delivered comm1 forum_shelter)
```

```
  ))

  (:metric minimize (total-cost))
)
```

problem-large.pddl

```
;; ============================================================================
;; problem-large.pddl : Instance 3 (large) - Tarragona chemical emergency
;; 3 drones spread over three regions (port, rambla, campus).
;; Scenario (loosely based on Tarragona): a leak at the petrochemical park.
;; Responders need a radio relay (comm1) at Bonavista before drones may enter
;; the park; the old town (Part Alta) stays closed until the Roman
;; amphitheatre below it is inspected for structural damage.
;; 12 locations, 2 depots (port, estacio), 3 charging stations,
;; battery capacity 6 (b0-b6), flight costs 1-3 levels per edge.
;; 8 goals: 4 deliveries + 4 inspections.
;; Generated from one edge list, so flight_cost and distance always match.
;; ============================================================================

(define (problem drone-large)
  (:domain drone_emergency)

  (:objects
    droneA droneB droneC - drone
    port serrallo bonavista petrochem ; west bank (port and industry)
    francoli ; river crossing
    estacio rambla amfiteatre part_alta ; city centre
    campus hospital pont_diable ; north
      - location
    comm1 - communication_device
    water1 - water_package
    kit1 kit2 - medical_kit
    b0 b1 b2 b3 b4 b5 b6 - level
  )

  (:init
    ;; ---- drones: full battery, empty cargo ----
    (drone_at droneA port) (battery droneA b6) (cargo_empty droneA)
    (drone_at droneB rambla) (battery droneB b6) (cargo_empty droneB)
    (drone_at droneC campus) (battery droneC b6) (cargo_empty droneC)

    ;; ---- map: every edge in both directions ----
    (connected port serrallo) (connected serrallo port)
    (connected port bonavista) (connected bonavista port)
    (connected serrallo bonavista) (connected bonavista serrallo)
    (connected bonavista petrochem) (connected petrochem bonavista)
    (connected port francoli) (connected francoli port)
    (connected francoli estacio) (connected estacio francoli)
    (connected hospital francoli) (connected francoli hospital)
    (connected estacio rambla) (connected rambla estacio)
    (connected rambla amfiteatre) (connected amfiteatre rambla)
    (connected amfiteatre part_alta) (connected part_alta amfiteatre)
    (connected rambla part_alta) (connected part_alta rambla)
    (connected rambla campus) (connected campus rambla)
    (connected campus hospital) (connected hospital campus)
    (connected campus pont_diable) (connected pont_diable campus)
    (connected bonavista campus) (connected campus bonavista)

    ;; ---- flight cost in battery levels ----
    (flight_cost port serrallo b1) (flight_cost serrallo port b1)
    (flight_cost port bonavista b2) (flight_cost bonavista port b2)
    (flight_cost serrallo bonavista b1) (flight_cost bonavista serrallo b1)
    (flight_cost bonavista petrochem b1) (flight_cost petrochem bonavista b1)
    (flight_cost port francoli b1) (flight_cost francoli port b1)
    (flight_cost francoli estacio b1) (flight_cost estacio francoli b1)
    (flight_cost hospital francoli b2) (flight_cost francoli hospital b2)
    (flight_cost estacio rambla b1) (flight_cost rambla estacio b1)
    (flight_cost rambla amfiteatre b1) (flight_cost amfiteatre rambla b1)
    (flight_cost amfiteatre part_alta b1) (flight_cost part_alta amfiteatre b1)
    (flight_cost rambla part_alta b2) (flight_cost part_alta rambla b2)
    (flight_cost rambla campus b2) (flight_cost campus rambla b2)
    (flight_cost campus hospital b1) (flight_cost hospital campus b1)
```

```
    (flight_cost campus pont_diable b2) (flight_cost pont_diable campus b2)
    (flight_cost bonavista campus b3) (flight_cost campus bonavista b3)

    ;; ---- distance for plan cost (same numbers as flight_cost) ----
    (= (distance port serrallo) 1) (= (distance serrallo port) 1)
    (= (distance port bonavista) 2) (= (distance bonavista port) 2)
    (= (distance serrallo bonavista) 1) (= (distance bonavista serrallo) 1)
    (= (distance bonavista petrochem) 1) (= (distance petrochem bonavista) 1)
    (= (distance port francoli) 1) (= (distance francoli port) 1)
    (= (distance francoli estacio) 1) (= (distance estacio francoli) 1)
    (= (distance hospital francoli) 2) (= (distance francoli hospital) 2)
    (= (distance estacio rambla) 1) (= (distance rambla estacio) 1)
    (= (distance rambla amfiteatre) 1) (= (distance amfiteatre rambla) 1)
    (= (distance amfiteatre part_alta) 1) (= (distance part_alta amfiteatre) 1)
    (= (distance rambla part_alta) 2) (= (distance part_alta rambla) 2)
    (= (distance rambla campus) 2) (= (distance campus rambla) 2)
    (= (distance campus hospital) 1) (= (distance hospital campus) 1)
    (= (distance campus pont_diable) 2) (= (distance pont_diable campus) 2)
    (= (distance bonavista campus) 3) (= (distance campus bonavista) 3)

    ;; ---- charging stations (bonavista is also a mission site) ----
    (has_charging_station bonavista) (has_charging_station rambla) (has_charging_station campus)

    ;; ---- access: everything open except petrochem and part_alta ----
    (accessible port) (accessible serrallo) (accessible bonavista) (accessible francoli)
    (accessible estacio) (accessible rambla) (accessible amfiteatre) (accessible campus)
    (accessible hospital) (accessible pont_diable)
    (unlocked_by_delivery comm1 petrochem)
    (unlocked_by_inspection amfiteatre part_alta)

    ;; ---- supplies at the depots and where they are needed ----
    (supply_at comm1 port)
    (supply_at water1 port)
    (supply_at kit1 estacio)
    (supply_at kit2 estacio)
    (needs_supply bonavista comm1)
    (needs_supply serrallo water1)
    (needs_supply part_alta kit1)
    (needs_supply hospital kit2)

    ;; ---- mission sites to inspect ----
    (needs_inspection francoli) (needs_inspection amfiteatre) (needs_inspection petrochem) (
        needs_inspection pont_diable)

    ;; ---- battery arithmetic (capacity 6, costs 1, 2, 3) ----
    ;; before - cost = left, only when before >= cost: 15 facts
    (battery_after b6 b1 b5) (battery_after b5 b1 b4) (battery_after b4 b1 b3)
    (battery_after b3 b1 b2) (battery_after b2 b1 b1) (battery_after b1 b1 b0)
    (battery_after b6 b2 b4) (battery_after b5 b2 b3) (battery_after b4 b2 b2)
    (battery_after b3 b2 b1) (battery_after b2 b2 b0)
    (battery_after b6 b3 b3) (battery_after b5 b3 b2) (battery_after b4 b3 b1)
    (battery_after b3 b3 b0)
    (next_level b0 b1) (next_level b1 b2) (next_level b2 b3) (next_level b3 b4) (next_level b4 b5) (
        next_level b5 b6)
    (max_level b6)

    (= (total-cost) 0)
  )

  (:goal (and
    (delivered comm1 bonavista)
    (delivered water1 serrallo)
    (delivered kit1 part_alta)
    (delivered kit2 hospital)
    (inspected francoli)
    (inspected amfiteatre)
    (inspected petrochem) ; locked until comm1 reaches bonavista
    (inspected pont_diable)
  ))

  (:metric minimize (total-cost))
)
```

problem-large-1drone.pddl

```
;; ============================================================================
;; problem-large-1drone.pddl : Instance 3 (large) - Tarragona chemical emergency
;; Single-drone variant: same map and goals, droneA only.
;; Scenario (loosely based on Tarragona): a leak at the petrochemical park.
;; Responders need a radio relay (comm1) at Bonavista before drones may enter
;; the park; the old town (Part Alta) stays closed until the Roman
;; amphitheatre below it is inspected for structural damage.
;; 12 locations, 2 depots (port, estacio), 3 charging stations,
;; battery capacity 6 (b0-b6), flight costs 1-3 levels per edge.
;; 8 goals: 4 deliveries + 4 inspections.
;; Generated from one edge list, so flight_cost and distance always match.
;; ============================================================================

(define (problem drone-large-1drone)
  (:domain drone_emergency)

  (:objects
    droneA - drone
    port serrallo bonavista petrochem ; west bank (port and industry)
    francoli ; river crossing
    estacio rambla amfiteatre part_alta ; city centre
    campus hospital pont_diable ; north
      - location
    comm1 - communication_device
    water1 - water_package
    kit1 kit2 - medical_kit
    b0 b1 b2 b3 b4 b5 b6 - level
  )

  (:init
    ;; ---- drones: full battery, empty cargo ----
    (drone_at droneA port) (battery droneA b6) (cargo_empty droneA)

    ;; ---- map: every edge in both directions ----
    (connected port serrallo) (connected serrallo port)
    (connected port bonavista) (connected bonavista port)
    (connected serrallo bonavista) (connected bonavista serrallo)
    (connected bonavista petrochem) (connected petrochem bonavista)
    (connected port francoli) (connected francoli port)
    (connected francoli estacio) (connected estacio francoli)
    (connected hospital francoli) (connected francoli hospital)
    (connected estacio rambla) (connected rambla estacio)
    (connected rambla amfiteatre) (connected amfiteatre rambla)
    (connected amfiteatre part_alta) (connected part_alta amfiteatre)
    (connected rambla part_alta) (connected part_alta rambla)
    (connected rambla campus) (connected campus rambla)
    (connected campus hospital) (connected hospital campus)
    (connected campus pont_diable) (connected pont_diable campus)
    (connected bonavista campus) (connected campus bonavista)

    ;; ---- flight cost in battery levels ----
    (flight_cost port serrallo b1) (flight_cost serrallo port b1)
    (flight_cost port bonavista b2) (flight_cost bonavista port b2)
    (flight_cost serrallo bonavista b1) (flight_cost bonavista serrallo b1)
    (flight_cost bonavista petrochem b1) (flight_cost petrochem bonavista b1)
    (flight_cost port francoli b1) (flight_cost francoli port b1)
    (flight_cost francoli estacio b1) (flight_cost estacio francoli b1)
    (flight_cost hospital francoli b2) (flight_cost francoli hospital b2)
    (flight_cost estacio rambla b1) (flight_cost rambla estacio b1)
    (flight_cost rambla amfiteatre b1) (flight_cost amfiteatre rambla b1)
    (flight_cost amfiteatre part_alta b1) (flight_cost part_alta amfiteatre b1)
    (flight_cost rambla part_alta b2) (flight_cost part_alta rambla b2)
    (flight_cost rambla campus b2) (flight_cost campus rambla b2)
    (flight_cost campus hospital b1) (flight_cost hospital campus b1)
    (flight_cost campus pont_diable b2) (flight_cost pont_diable campus b2)
    (flight_cost bonavista campus b3) (flight_cost campus bonavista b3)

    ;; ---- distance for plan cost (same numbers as flight_cost) ----
    (= (distance port serrallo) 1) (= (distance serrallo port) 1)
    (= (distance port bonavista) 2) (= (distance bonavista port) 2)
    (= (distance serrallo bonavista) 1) (= (distance bonavista serrallo) 1)
    (= (distance bonavista petrochem) 1) (= (distance petrochem bonavista) 1)
```

```
    (= (distance port francoli) 1) (= (distance francoli port) 1)
    (= (distance francoli estacio) 1) (= (distance estacio francoli) 1)
    (= (distance hospital francoli) 2) (= (distance francoli hospital) 2)
    (= (distance estacio rambla) 1) (= (distance rambla estacio) 1)
    (= (distance rambla amfiteatre) 1) (= (distance amfiteatre rambla) 1)
    (= (distance amfiteatre part_alta) 1) (= (distance part_alta amfiteatre) 1)
    (= (distance rambla part_alta) 2) (= (distance part_alta rambla) 2)
    (= (distance rambla campus) 2) (= (distance campus rambla) 2)
    (= (distance campus hospital) 1) (= (distance hospital campus) 1)
    (= (distance campus pont_diable) 2) (= (distance pont_diable campus) 2)
    (= (distance bonavista campus) 3) (= (distance campus bonavista) 3)

    ;; ---- charging stations (bonavista is also a mission site) ----
    (has_charging_station bonavista) (has_charging_station rambla) (has_charging_station campus)

    ;; ---- access: everything open except petrochem and part_alta ----
    (accessible port) (accessible serrallo) (accessible bonavista) (accessible francoli)
    (accessible estacio) (accessible rambla) (accessible amfiteatre) (accessible campus)
    (accessible hospital) (accessible pont_diable)
    (unlocked_by_delivery comm1 petrochem)
    (unlocked_by_inspection amfiteatre part_alta)

    ;; ---- supplies at the depots and where they are needed ----
    (supply_at comm1 port)
    (supply_at water1 port)
    (supply_at kit1 estacio)
    (supply_at kit2 estacio)
    (needs_supply bonavista comm1)
    (needs_supply serrallo water1)
    (needs_supply part_alta kit1)
    (needs_supply hospital kit2)

    ;; ---- mission sites to inspect ----
    (needs_inspection francoli) (needs_inspection amfiteatre) (needs_inspection petrochem) (
        needs_inspection pont_diable)

    ;; ---- battery arithmetic (capacity 6, costs 1, 2, 3) ----
    ;; before - cost = left, only when before >= cost: 15 facts
    (battery_after b6 b1 b5) (battery_after b5 b1 b4) (battery_after b4 b1 b3)
    (battery_after b3 b1 b2) (battery_after b2 b1 b1) (battery_after b1 b1 b0)
    (battery_after b6 b2 b4) (battery_after b5 b2 b3) (battery_after b4 b2 b2)
    (battery_after b3 b2 b1) (battery_after b2 b2 b0)
    (battery_after b6 b3 b3) (battery_after b5 b3 b2) (battery_after b4 b3 b1)
    (battery_after b3 b3 b0)
    (next_level b0 b1) (next_level b1 b2) (next_level b2 b3) (next_level b3 b4) (next_level b4 b5) (
        next_level b5 b6)
    (max_level b6)

    (= (total-cost) 0)
  )

  (:goal (and
    (delivered comm1 bonavista)
    (delivered water1 serrallo)
    (delivered kit1 part_alta)
    (delivered kit2 hospital)
    (inspected francoli)
    (inspected amfiteatre)
    (inspected petrochem) ; locked until comm1 reaches bonavista
    (inspected pont_diable)
  ))

  (:metric minimize (total-cost))
)
```

A.3 Example plans

Optimal plans (A* + LM-cut) under Model B, and the single-drone plan under Model A. The Model A plans of the three instances appear inside the VAL outputs below.

small, Model B

```
(pick-up-supply dronea kit1 aerodrome)
(fly dronea aerodrome hospital_viladecans b4 b1 b3)
(deliver-supply dronea kit1 hospital_viladecans)
(fly dronea hospital_viladecans aerodrome b3 b1 b2)
(fly dronea aerodrome barceloneta b2 b1 b1)
(inspect dronea barceloneta)
(fly dronea barceloneta montjuic_pad b1 b1 b0)
(recharge dronea montjuic_pad b0 b1)
(fly dronea montjuic_pad ciutadella b1 b1 b0)
(inspect dronea ciutadella)
; cost = 10 (unit cost)
```

medium, Model B

```
(pick-up-supply dronea comm1 port_depot)
(fly dronea port_depot barceloneta b5 b1 b4)
(inspect dronea barceloneta)
(fly dronea barceloneta hospital_mar b4 b1 b3)
(inspect dronea hospital_mar)
(fly dronea hospital_mar poblenou b3 b1 b2)
(inspect dronea poblenou)
(fly dronea poblenou besos_delta b2 b1 b1)
(inspect dronea besos_delta)
(fly dronea besos_delta forum_shelter b1 b1 b0)
(deliver-supply dronea comm1 forum_shelter)
(recharge droneb glories_tower b4 b5)
(recharge droneb glories_tower b5 b6)
(fly droneb glories_tower port_depot b6 b2 b4)
(pick-up-supply droneb water1 port_depot)
(fly droneb port_depot glories_tower b4 b2 b2)
(recharge droneb glories_tower b2 b3)
(recharge droneb glories_tower b3 b4)
(recharge droneb glories_tower b4 b5)
(recharge droneb glories_tower b5 b6)
(fly droneb glories_tower poblenou b6 b1 b5)
(deliver-supply droneb water1 poblenou)
(fly droneb poblenou hospital_mar b5 b1 b4)
(fly droneb hospital_mar barceloneta b4 b1 b3)
(fly droneb barceloneta port_depot b3 b1 b2)
(pick-up-supply droneb kit1 port_depot)
(fly droneb port_depot barceloneta b2 b1 b1)
(fly droneb barceloneta hospital_mar b1 b1 b0)
(deliver-supply droneb kit1 hospital_mar)
; cost = 31 (general cost)
```

large, Model B

```
(fly droneb rambla estacio b6 b1 b5)
(pick-up-supply droneb kit2 estacio)
(fly droneb estacio francoli b5 b1 b4)
(inspect droneb francoli)
(fly droneb francoli hospital b4 b2 b2)
(deliver-supply droneb kit2 hospital)
(fly dronec campus rambla b6 b2 b4)
(fly dronec rambla estacio b4 b1 b3)
(pick-up-supply dronec kit1 estacio)
(fly dronec estacio rambla b3 b1 b2)
(pick-up-supply dronea water1 port)
(fly dronec rambla amfiteatre b2 b1 b1)
(inspect dronec amfiteatre)
(fly dronec amfiteatre part_alta b1 b1 b0)
(deliver-supply dronec kit1 part_alta)
(fly dronea port serrallo b6 b1 b5)
(deliver-supply dronea water1 serrallo)
(fly dronea serrallo port b5 b1 b4)
(pick-up-supply dronea comm1 port)
(fly dronea port bonavista b4 b2 b2)
(deliver-supply dronea comm1 bonavista)
(fly dronea bonavista petrochem b2 b1 b1)
(inspect dronea petrochem)
(fly droneb hospital campus b2 b1 b1)
(recharge droneb campus b1 b2)
(fly droneb campus pont_diable b2 b2 b0)
(inspect droneb pont_diable)
; cost = 31 (general cost)
```

large-1drone, Model A

```
(pick-up-supply dronea water1 port)
(fly dronea port serrallo b6 b1 b5)
(deliver-supply dronea water1 serrallo)
(fly dronea serrallo port b5 b1 b4)
(pick-up-supply dronea comm1 port)
(fly dronea port bonavista b4 b2 b2)
(deliver-supply dronea comm1 bonavista)
(fly dronea bonavista petrochem b2 b1 b1)
(inspect dronea petrochem)
(fly dronea petrochem bonavista b1 b1 b0)
(recharge dronea bonavista b0 b1 b6)
(fly dronea bonavista port b6 b2 b4)
(fly dronea port francoli b4 b1 b3)
(inspect dronea francoli)
(fly dronea francoli estacio b3 b1 b2)
(pick-up-supply dronea kit1 estacio)
(fly dronea estacio rambla b2 b1 b1)
(recharge dronea rambla b1 b2 b6)
(fly dronea rambla amfiteatre b6 b1 b5)
(inspect dronea amfiteatre)
(fly dronea amfiteatre part_alta b5 b1 b4)
(deliver-supply dronea kit1 part_alta)
(fly dronea part_alta rambla b4 b2 b2)
(recharge dronea rambla b2 b3 b6)
(fly dronea rambla estacio b6 b1 b5)
(pick-up-supply dronea kit2 estacio)
(fly dronea estacio francoli b5 b1 b4)
(fly dronea francoli hospital b4 b2 b2)
(deliver-supply dronea kit2 hospital)
(fly dronea hospital campus b2 b1 b1)
(recharge dronea campus b1 b2 b6)
(fly dronea campus pont_diable b6 b2 b4)
(inspect dronea pont_diable)
; cost = 38 (general cost)
```

A.4 VAL outputs

One full `Validate -v` output per instance (Model A, A* + LM-cut). Each one ends with `Plan valid` and the final value of `total-cost`.

small, Model A

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
Checking plan: plans/problem-small_A_astar-lmcut.plan
Plan to validate:

Plan size: 10
1:
(pick-up-supply dronea kit1 aerodrome)

2:
(fly dronea aerodrome hospital_viladecans b4 b1 b3)

3:
(deliver-supply dronea kit1 hospital_viladecans)

4:
(fly dronea hospital_viladecans aerodrome b3 b1 b2)

5:
(fly dronea aerodrome barceloneta b2 b1 b1)

6:
(inspect dronea barceloneta)

7:
(fly dronea barceloneta montjuic_pad b1 b1 b0)

8:
(recharge dronea montjuic_pad b0 b1 b4)

9:
(fly dronea montjuic_pad ciutadella b4 b1 b3)

10:
(inspect dronea ciutadella)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply_at kit1 aerodrome)
Deleting (cargo_empty dronea)
Adding (carrying dronea kit1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (drone_at dronea aerodrome)
Deleting (battery dronea b4)
Adding (drone_at dronea hospital_viladecans)
Adding (battery dronea b3)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (carrying dronea kit1)
Deleting (needs_supply hospital_viladecans kit1)
Adding (cargo_empty dronea)
Adding (delivered kit1 hospital_viladecans)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Deleting (drone_at dronea hospital_viladecans)
Deleting (battery dronea b3)
Adding (drone_at dronea aerodrome)
Adding (battery dronea b2)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Deleting (drone_at dronea aerodrome)
Deleting (battery dronea b2)
Adding (drone_at dronea barceloneta)
Adding (battery dronea b1)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
Deleting (needs_inspection barceloneta)
Adding (inspected barceloneta)
Adding (accessible ciutadella)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 7)
Deleting (drone_at dronea barceloneta)
Deleting (battery dronea b1)
Adding (drone_at dronea montjuic_pad)
Adding (battery dronea b0)
Updating (total-cost) (6) by 1 increase

Checking next happening (time 8)
Deleting (battery dronea b0)
Adding (battery dronea b4)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 9)
Deleting (drone_at dronea montjuic_pad)
Deleting (battery dronea b4)
Adding (drone_at dronea ciutadella)
Adding (battery dronea b3)
Updating (total-cost) (8) by 1 increase

Checking next happening (time 10)
Deleting (needs_inspection ciutadella)
Adding (inspected ciutadella)
Updating (total-cost) (9) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 10

Successful plans:
Value: 10
 plans/problem-small_A_astar-lmcut.plan 10
```

medium, Model A

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
```

```
Checking plan: plans/problem-medium_A_astar-lmcut.plan
Plan to validate:

Plan size: 24
1:
(pick-up-supply dronea comm1 port_depot)

2:
(fly dronea port_depot barceloneta b5 b1 b4)

3:
(inspect dronea barceloneta)

4:
(fly dronea barceloneta hospital_mar b4 b1 b3)

5:
(inspect dronea hospital_mar)

6:
(fly dronea hospital_mar poblenou b3 b1 b2)

7:
(inspect dronea poblenou)

8:
(fly dronea poblenou besos_delta b2 b1 b1)

9:
(inspect dronea besos_delta)

10:
(fly dronea besos_delta forum_shelter b1 b1 b0)

11:
(deliver-supply dronea comm1 forum_shelter)

12:
(fly droneb glories_tower port_depot b4 b2 b2)

13:
(pick-up-supply droneb water1 port_depot)

14:
(fly droneb port_depot glories_tower b2 b2 b0)

15:
(recharge droneb glories_tower b0 b1 b6)

16:
(fly droneb glories_tower poblenou b6 b1 b5)

17:
(deliver-supply droneb water1 poblenou)

18:
(fly droneb poblenou hospital_mar b5 b1 b4)

19:
(fly droneb hospital_mar barceloneta b4 b1 b3)

20:
(fly droneb barceloneta port_depot b3 b1 b2)

21:
(pick-up-supply droneb kit1 port_depot)

22:
(fly droneb port_depot barceloneta b2 b1 b1)

23:
```

```
(fly droneb barceloneta hospital_mar b1 b1 b0)

24:
(deliver-supply droneb kit1 hospital_mar)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply_at comm1 port_depot)
Deleting (cargo_empty dronea)
Adding (carrying dronea comm1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (drone_at dronea port_depot)
Deleting (battery dronea b5)
Adding (drone_at dronea barceloneta)
Adding (battery dronea b4)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (needs_inspection barceloneta)
Adding (inspected barceloneta)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Deleting (drone_at dronea barceloneta)
Deleting (battery dronea b4)
Adding (drone_at dronea hospital_mar)
Adding (battery dronea b3)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Deleting (needs_inspection hospital_mar)
Adding (inspected hospital_mar)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
Deleting (drone_at dronea hospital_mar)
Deleting (battery dronea b3)
Adding (drone_at dronea poblenou)
Adding (battery dronea b2)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 7)
Deleting (needs_inspection poblenou)
Adding (inspected poblenou)
Adding (accessible besos_delta)
Updating (total-cost) (6) by 1 increase

Checking next happening (time 8)
Deleting (drone_at dronea poblenou)
Deleting (battery dronea b2)
Adding (drone_at dronea besos_delta)
Adding (battery dronea b1)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 9)
Deleting (needs_inspection besos_delta)
Adding (inspected besos_delta)
Updating (total-cost) (8) by 1 increase

Checking next happening (time 10)
Deleting (drone_at dronea besos_delta)
Deleting (battery dronea b1)
Adding (drone_at dronea forum_shelter)
Adding (battery dronea b0)
Updating (total-cost) (9) by 1 increase

Checking next happening (time 11)
Deleting (carrying dronea comm1)
Deleting (needs_supply forum_shelter comm1)
Adding (cargo_empty dronea)
Adding (delivered comm1 forum_shelter)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 12)
Deleting (drone_at droneb glories_tower)
Deleting (battery droneb b4)
Adding (drone_at droneb port_depot)
Adding (battery droneb b2)
Updating (total-cost) (11) by 2 increase

Checking next happening (time 13)
Deleting (supply_at water1 port_depot)
Deleting (cargo_empty droneb)
Adding (carrying droneb water1)
Updating (total-cost) (13) by 1 increase

Checking next happening (time 14)
Deleting (drone_at droneb port_depot)
Deleting (battery droneb b2)
Adding (drone_at droneb glories_tower)
Adding (battery droneb b0)
Updating (total-cost) (14) by 2 increase

Checking next happening (time 15)
Deleting (battery droneb b0)
Adding (battery droneb b6)
Updating (total-cost) (16) by 1 increase

Checking next happening (time 16)
Deleting (drone_at droneb glories_tower)
Deleting (battery droneb b6)
Adding (drone_at droneb poblenou)
Adding (battery droneb b5)
Updating (total-cost) (17) by 1 increase

Checking next happening (time 17)
Deleting (carrying droneb water1)
Deleting (needs_supply poblenou water1)
Adding (cargo_empty droneb)
Adding (delivered water1 poblenou)
Updating (total-cost) (18) by 1 increase

Checking next happening (time 18)
Deleting (drone_at droneb poblenou)
Deleting (battery droneb b5)
Adding (drone_at droneb hospital_mar)
Adding (battery droneb b4)
Updating (total-cost) (19) by 1 increase

Checking next happening (time 19)
Deleting (drone_at droneb hospital_mar)
Deleting (battery droneb b4)
Adding (drone_at droneb barceloneta)
Adding (battery droneb b3)
Updating (total-cost) (20) by 1 increase

Checking next happening (time 20)
Deleting (drone_at droneb barceloneta)
Deleting (battery droneb b3)
Adding (drone_at droneb port_depot)
Adding (battery droneb b2)
Updating (total-cost) (21) by 1 increase

Checking next happening (time 21)
Deleting (supply_at kit1 port_depot)
Deleting (cargo_empty droneb)
Adding (carrying droneb kit1)
Updating (total-cost) (22) by 1 increase

Checking next happening (time 22)
Deleting (drone_at droneb port_depot)
Deleting (battery droneb b2)
Adding (drone_at droneb barceloneta)
Adding (battery droneb b1)
Updating (total-cost) (23) by 1 increase

Checking next happening (time 23)
Deleting (drone_at droneb barceloneta)
Deleting (battery droneb b1)
Adding (drone_at droneb hospital_mar)
Adding (battery droneb b0)
Updating (total-cost) (24) by 1 increase

Checking next happening (time 24)
Deleting (carrying droneb kit1)
Deleting (needs_supply hospital_mar kit1)
Adding (cargo_empty droneb)
Adding (delivered kit1 hospital_mar)
Updating (total-cost) (25) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 26

Successful plans:
Value: 26
 plans/problem-medium_A_astar-lmcut.plan 26
```

large, Model A

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
```

```
Checking plan: plans/problem-large_A_astar-lmcut.plan
Plan to validate:

Plan size: 26
1:
(pick-up-supply dronea water1 port)

2:
(fly droneb rambla estacio b6 b1 b5)

3:
(fly dronec campus pont_diable b6 b2 b4)

4:
(pick-up-supply droneb kit1 estacio)

5:
(fly droneb estacio rambla b5 b1 b4)

6:
(fly droneb rambla amfiteatre b4 b1 b3)

7:
(inspect droneb amfiteatre)

8:
(fly droneb amfiteatre part_alta b3 b1 b2)

9:
(deliver-supply droneb kit1 part_alta)

10:
(inspect dronec pont_diable)

11:
(fly droneb part_alta rambla b2 b2 b0)

12:
```

```
(recharge droneb rambla b0 b1 b6)

13:
(fly droneb rambla estacio b6 b1 b5)

14:
(pick-up-supply droneb kit2 estacio)

15:
(fly droneb estacio francoli b5 b1 b4)

16:
(inspect droneb francoli)

17:
(fly droneb francoli hospital b4 b2 b2)

18:
(deliver-supply droneb kit2 hospital)

19:
(fly dronea port serrallo b6 b1 b5)

20:
(deliver-supply dronea water1 serrallo)

21:
(fly dronea serrallo port b5 b1 b4)

22:
(pick-up-supply dronea comm1 port)

23:
(fly dronea port bonavista b4 b2 b2)

24:
(deliver-supply dronea comm1 bonavista)

25:
(fly dronea bonavista petrochem b2 b1 b1)

26:
(inspect dronea petrochem)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply_at water1 port)
Deleting (cargo_empty dronea)
Adding (carrying dronea water1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (drone_at droneb rambla)
Deleting (battery droneb b6)
Adding (drone_at droneb estacio)
Adding (battery droneb b5)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (drone_at dronec campus)
Deleting (battery dronec b6)
Adding (drone_at dronec pont_diable)
Adding (battery dronec b4)
Updating (total-cost) (2) by 2 increase

Checking next happening (time 4)
Deleting (supply_at kit1 estacio)
Deleting (cargo_empty droneb)
Adding (carrying droneb kit1)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 5)
Deleting (drone_at droneb estacio)
Deleting (battery droneb b5)
Adding (drone_at droneb rambla)
Adding (battery droneb b4)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 6)
Deleting (drone_at droneb rambla)
Deleting (battery droneb b4)
Adding (drone_at droneb amfiteatre)
Adding (battery droneb b3)
Updating (total-cost) (6) by 1 increase

Checking next happening (time 7)
Deleting (needs_inspection amfiteatre)
Adding (inspected amfiteatre)
Adding (accessible part_alta)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 8)
Deleting (drone_at droneb amfiteatre)
Deleting (battery droneb b3)
Adding (drone_at droneb part_alta)
Adding (battery droneb b2)
Updating (total-cost) (8) by 1 increase

Checking next happening (time 9)
Deleting (carrying droneb kit1)
Deleting (needs_supply part_alta kit1)
Adding (cargo_empty droneb)
Adding (delivered kit1 part_alta)
Updating (total-cost) (9) by 1 increase

Checking next happening (time 10)
Deleting (needs_inspection pont_diable)
Adding (inspected pont_diable)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 11)
Deleting (drone_at droneb part_alta)
Deleting (battery droneb b2)
Adding (drone_at droneb rambla)
Adding (battery droneb b0)
Updating (total-cost) (11) by 2 increase

Checking next happening (time 12)
Deleting (battery droneb b0)
Adding (battery droneb b6)
Updating (total-cost) (13) by 1 increase

Checking next happening (time 13)
Deleting (drone_at droneb rambla)
Deleting (battery droneb b6)
Adding (drone_at droneb estacio)
Adding (battery droneb b5)
Updating (total-cost) (14) by 1 increase

Checking next happening (time 14)
Deleting (supply_at kit2 estacio)
Deleting (cargo_empty droneb)
Adding (carrying droneb kit2)
Updating (total-cost) (15) by 1 increase

Checking next happening (time 15)
Deleting (drone_at droneb estacio)
Deleting (battery droneb b5)
Adding (drone_at droneb francoli)
Adding (battery droneb b4)
Updating (total-cost) (16) by 1 increase

Checking next happening (time 16)
Deleting (needs_inspection francoli)
Adding (inspected francoli)
Updating (total-cost) (17) by 1 increase

Checking next happening (time 17)
Deleting (drone_at droneb francoli)
Deleting (battery droneb b4)
Adding (drone_at droneb hospital)
Adding (battery droneb b2)
Updating (total-cost) (18) by 2 increase

Checking next happening (time 18)
Deleting (carrying droneb kit2)
Deleting (needs_supply hospital kit2)
Adding (cargo_empty droneb)
Adding (delivered kit2 hospital)
Updating (total-cost) (20) by 1 increase

Checking next happening (time 19)
Deleting (drone_at dronea port)
Deleting (battery dronea b6)
Adding (drone_at dronea serrallo)
Adding (battery dronea b5)
Updating (total-cost) (21) by 1 increase

Checking next happening (time 20)
Deleting (carrying dronea water1)
Deleting (needs_supply serrallo water1)
Adding (cargo_empty dronea)
Adding (delivered water1 serrallo)
Updating (total-cost) (22) by 1 increase

Checking next happening (time 21)
Deleting (drone_at dronea serrallo)
Deleting (battery dronea b5)
Adding (drone_at dronea port)
Adding (battery dronea b4)
Updating (total-cost) (23) by 1 increase

Checking next happening (time 22)
Deleting (supply_at comm1 port)
Deleting (cargo_empty dronea)
Adding (carrying dronea comm1)
Updating (total-cost) (24) by 1 increase

Checking next happening (time 23)
Deleting (drone_at dronea port)
Deleting (battery dronea b4)
Adding (drone_at dronea bonavista)
Adding (battery dronea b2)
Updating (total-cost) (25) by 2 increase
```

```
Checking next happening (time 24)
Deleting (carrying dronea comm1)
Deleting (needs_supply bonavista comm1)
Adding (cargo_empty dronea)
Adding (delivered comm1 bonavista)
Adding (accessible petrochem)
Updating (total-cost) (27) by 1 increase

Checking next happening (time 25)
Deleting (drone_at dronea bonavista)
Deleting (battery dronea b2)
Adding (drone_at dronea petrochem)
Adding (battery dronea b1)
Updating (total-cost) (28) by 1 increase

Checking next happening (time 26)
Deleting (needs_inspection petrochem)
Adding (inspected petrochem)
Updating (total-cost) (29) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 30

Successful plans:
Value: 30
 plans/problem-large_A_astar-lmcut.plan 30
```